Gwave — System Design & Architecture

SYSTEM DOCUMENTATION

Gwave
System Design & Architecture

Project plan, functional design, AWS infrastructure and database reference for the gwave.cc super-app and its native Android client.

Product	Gwave (GreenWave) — https://gwave.cc
Repository	KoNyein/gwave.ai
Document	Revision 1.0 · 01 August 2026
Status	Live production system
Audience	Engineering, operations, investors, partners

159 DATABASE TABLES

111 API ROUTES

130+ WEB PAGES

2 CLIENT APPS

Generated from the source repository — every figure reflects code in `main`.

Confidential

Gwave — System Design & Architecture

Contents

This document was assembled directly from the repository: route counts, table names, foreign keys, workflow steps and environment variables were read out of the code and SQL rather than transcribed from memory. Where a design decision has a non-obvious reason, the reason is recorded next to it — those notes are the part of a system that is normally lost.

Gwave — System Design & Architecture

1 Executive summary

Gwave is a production social super-app for Myanmar users: a single Next.js 14 application and a native Flutter Android client, running on an all-AWS backend, serving real users today at `https://gwave.cc`.

What began as a Facebook-style social network for growers has become a multi-domain platform. The social core — feed, groups, pages, messenger, stories, reels — is only one of six product families. Alongside it run live streaming, a wallet and commerce stack, ride hailing, a nine-track learning platform, IoT and CCTV monitoring, and a public safety layer. All of it shares one identity system, one database, one deployment pipeline and one release channel, which is the single most important architectural property of the project: *one app, one pipeline, one release*.

159
TABLES ON RDS, RLS ON EVERY ONE

241
FOREIGN-KEY RELATIONSHIPS

~200
SQL FUNCTIONS AND TRIGGERS

93
MIGRATIONS + 48 PATCH FILES

1.1 What the system is

Table 1 — System identity at a glance

Dimension	Description
Product	Gwave — social super-app combining communication, commerce, learning, mobility, sensing and safety in one account.
Web client	Next.js 14 App Router, TypeScript strict, Tailwind + shadcn/ui, installable PWA. 130+ pages.
Mobile client	Native Flutter Android app (`mobile/`, 141 Dart files, 38 feature modules), distributed as a signed APK from `gwave.cc/welcome`.
Backend	PostgreSQL on Amazon RDS, exposed by self-hosted PostgREST and a self-hosted Realtime server; S3 + CloudFront storage; Amazon Cognito identity.
Hosting	Self-hosted on AWS EC2 (ap-southeast-1). Next.js standalone image → ECR → EC2 via SSM. No Vercel, no managed PaaS.
Media plane	AWS IVS (low-latency + real-time), LiveKit Cloud SFU, coturn TURN relay, Kinesis Video Streams for CCTV.
Languages	Burmese, English, Thai (`my` / `en` / `th`) with runtime switching and English fallback.

1.2 Design principles

Six rules govern how the system is extended. They exist because each was learned from a production failure, and they are enforced in code review, in the repository's own instructions file, and in the deployment scripts.

One app, one pipeline, one release
The mobile app is `mobile/`, built only by `build-flutter-apk.yml`, published only to the rolling `mobile-latest` release. A second app, workflow or channel is never scaffolded — divergence, not scale, is what kills a small team.

Visible state first, side effects second
A broadcast is marked live *before* any provider call. Recording and restreaming are follow-up updates that may fail without costing anything a person can see. Being live is not a side effect of recording.

Gwave — System Design & Architecture

Never guess at identity

`custom:profile_id` is the authoritative account link. Helpers return `undefined` for failure and `null` for absent, and a back-fill is only allowed on `null`. Violating this once destroyed a user's account.

No embeds on hot paths

PostgREST resource embeds die silently behind a stale schema cache. Hot-path client queries are flat; joins are assembled in application code. Messenger, notifications and call verification were all lost to this once.

Data fixes shipped apps

A shipped APK cannot be patched, but the rows it reads can. Sweepers that correct stale state server-side are treated as first-class features, not cleanup jobs.

Money moves only through RPCs

Balances are trigger-protected columns; every transfer is a `SECURITY DEFINER` function with `EXECUTE` granted to the service role alone. No client writes a balance.

1.3 Current status

Table 2 — Subsystem readiness

Subsystem	Status	Notes
Web application	**Live**	main auto-deploys to gwave.cc through ECR + SSM.
Android app	**Live**	Signed APK on every push to `mobile/**`; in-app update banner; build number visible in Settings.
Calls (WebRTC)	**Live**	coturn TURN relay on the app host; ring inbox, reconnect, callee push.
Live streaming	**Live**	Browser → LiveKit/IVS stage with restream to a low-latency channel; app → IVS RTMPS; replays linked by a one-minute sweeper.
Ride hailing	**Live**	Passenger and driver modes, dispatcher, in-trip SOS, public trip-share link.
G-Pay wallet	**Live**	Top-up, transfer, POS settlement, ride commission settlement, PIN + phone OTP + face KYC.
Smart farm / CCTV	**Live**	MQTT bridge, automation rules, camera HLS and KVS WebRTC.
FCM push	**Gap**	Calls and messages do not ring when the app is closed; web-push covers open browsers only. Needs a Firebase project.
LiveKit egress	**Gap**	Browser Go Live sessions get no replay until static IAM keys for egress are configured.
Native iOS	**Blocked**	Requires Apple Developer Program enrolment. PWA install guide is the interim path.

Gwave — System Design & Architecture

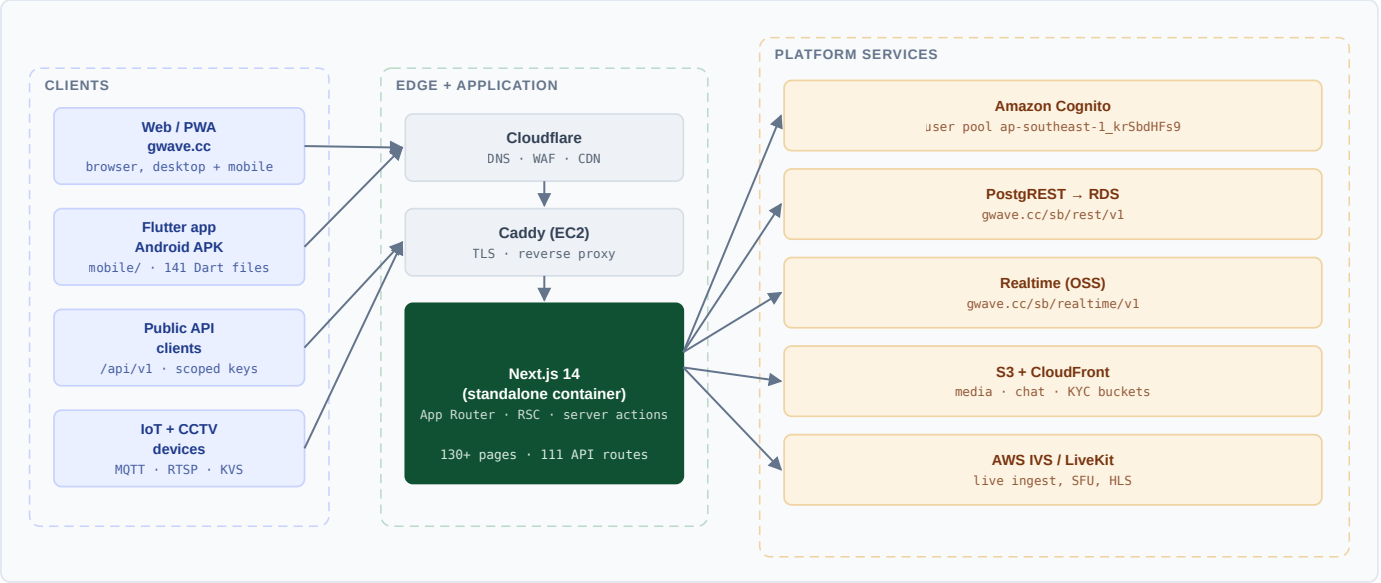

Figure 1 — System context. Four client classes reach one application tier, which fans out to platform services that are all under the project's own AWS account.

Gwave — System Design & Architecture

2 Product scope and feature catalogue

Gwave is organised into six product families. Each family owns its own routes, its own database tables and its own screens in both clients, but shares the platform's identity, wallet, notification and media services.

Figure 2 — Product families and the capability groups inside them. Nothing in the platform exists twice: the wallet that pays for a shop order is the wallet that settles a driver's commission.

2.1 Social core

Table 3 — Social core — routes and capabilities

Capability	Routes	Detail
News feed	`/feed, /p/[id]`	Keyset-paginated feed, text and media posts, four visibility levels, boost injection, live and story rail, muted autoplay previews.
Reactions & comments	`/p/[id]`	Multi-type reactions on posts and comments, nested comments with enforced depth, photo comments, counters maintained by triggers.
Sharing	`/feed`	Share creates a second post referencing the original; counters and notifications fire through `handle_shared_post()`.
Profiles	`/u/[username], /profile`	Cover and avatar, bio, status, photos tab, reviews, demographics, presence dot from `last_seen_at`.
Graph	`/friends`	Friend requests (requester/addressee) and asymmetric follows, both enforced in RLS via `are_friends()`.
Groups & pages	`/groups, /pages`	Group roles and member counts, business pages with followers, group-scoped posts and camera shares.
Stories	`/feed`	24-hour stories with view tracking and a cleanup job.

Gwave — System Design & Architecture

Capability	Routes	Detail
Reels	`/reels, /reels/analytics`	Vertical video, watch-time accounting, likes and views, creator earnings and withdrawal.
Notifications	`/notifications`	Trigger-generated rows for reactions, comments, shares, follows, friendships and alerts; realtime bell plus web push.
Moderation	`/admin/moderation`	Reports on posts, comments and profiles; blocks; suspension; audit log of every admin action.

2.2 Communication

Table 4 — Communication — routes and capabilities

Capability	Routes	Detail
Messenger	`/messages`	Direct and group conversations, typing indicators, read receipts, image and voice messages, attachments through a private bucket, link previews.
Audio / video calls	`/messages, /api/webrtc/ice`	WebRTC with Realtime broadcast signaling: `calls:{userId}` ring inbox and `call:{callId}` per-call channel; TURN relay for carrier NAT; every call logged once.
Push-to-talk	`/talk, /talk/[id]`	Channel-based PTT with membership control and message history.
Live streaming	`/live, /live/[id], /live/new`	Browser Go Live and in-app RTMPS broadcast, HLS playback everywhere, chat, floating reactions, gifts, presence viewer count, replays.
Co-host	`/live/cohost, /live/cohost/[code]`	Invite guests into a LiveKit room; mesh fallback when no SFU is configured.
Meetings	`/meet, /meet/[room]`	Ad-hoc rooms for classroom and group use.
Offline chat	`app only`	Nearby Connections peer-to-peer messaging over Bluetooth/Wi-Fi with GPS sharing, for use with no internet at all.

2.3 Commerce and money

Table 5 — Commerce — routes and capabilities

Capability	Routes	Detail
G-Pay wallet	`/gpay, /finance`	KYC-gated accounts, balance, transfers, Stripe top-up, admin slip top-up, currency conversion, PIN and phone OTP, face KYC in a private bucket.
Shop	`/shop, /shop/[id], /shop/sell`	Dropship and affiliate listings, AliExpress import with full galleries, specs, variants and merchant ratings; seller customer-pack for resale.
Orders	`/shop/orders, /shop/sales`	Order lifecycle with delivery milestones, buyer push on status change, seller dashboard.
Point of sale	`/pos/*`	Multi-store POS: products, categories, inventory and stock movements, shifts, atomic sale and refund RPCs, 80 mm receipts, reports, CSV export, G-Pay settlement.

Gwave — System Design & Architecture

Capability	Routes	Detail
Membership	`/membership`	Stripe Checkout and PromptPay QR with slip review, role synchronisation, members-only content and tools.
Boosts (ads)	`/boost, /boost/new`	Escrowed ad spend, auction-based feed injection, impression and click accounting, daily statistics.
Jobs	`/jobs/*`	Employer postings, applications, management console.
Marketplace & dating	`app + /api/mobile/*`	Classified listings and a swipe/match dating module sharing the same profile identity.

2.4 Knowledge and learning

Table 6 — Knowledge — routes and capabilities

Capability	Routes	Detail
Strain encyclopedia	`/strains, /strains/[slug]`	200 seeded strains, full-text plus trigram search, inline comments with photos, adult gate.
Mineral encyclopedia	`/minerals, /minerals/[slug]`	100 minerals with Myanmar-specific regional and legal notes.
Mine site map	`/minerals/mines`	Community-submitted pins for 16 minerals, complete-or-rejected validation in both zod and CHECK constraints, report and admin moderation.
Metal & cannabis markets	`/metals, /strains/market`	Live LME/COMEX reference prices, hand-recorded border and local quotes per gate, Thai equity board with a compliance panel.
Cannabis place map	`/strains/places`	Community listings with photo requirements, reports and an admin queue.
Learn platform	`/learn/*`	Nine tracks of 60 lessons (Python, JavaScript, HTML, CSS, SQL, AI, Electronics/IoT, Robotics, Applied Agri Science), runnable playgrounds (Pyodide, sql.js, CodeMirror), quizzes, progress, projects, certificates and a leaderboard.
Live classrooms	`/learn/live, /learn/teach`	Teacher applications and admin approval; approved teachers stream lessons on the live stack.
Languages & typing	`/learn/languages, /learn/languages/typing`	Language units and a typing tutor with recorded scores.
Games	`/games/*`	Curated game catalogue, community uploads in a sandboxed frame, reactions and comments, drone simulator, grow game.
Grower tools	`/tools/*`	EC/PPM, VPD, unit and currency converters, profit calculator, QR tool; member-only nutrient mixing and yield estimator; results shareable as posts.

2.5 Life services

Table 7 — Life services — routes and capabilities

Capability	Routes	Detail
Ride hailing (rider)	`/api/ride/*, app`	One map screen with a state-driven bottom sheet, quote with labelled surge, 3-second poll that doubles as the dispatcher clock, live driver position, trip resumption after an app kill.

Gwave — System Design & Architecture

Capability	Routes	Detail
Ride hailing (driver)	`/api/ride/driver/*`	Application with three documents, online switch backed by a process-wide 30-second heartbeat, offer countdown, one-button trip progression, earnings and commission owed, G-Pay settlement.
Trip safety	`/ride/track/[token]`	In-trip SOS carrying plate, vehicle, driver and destination; unguessable public trip-share page that deliberately withholds rider identity, phone numbers, fare and payment method; `noindex`; expires 30 minutes after arrival.
Family	`/family, /family/trackers`	Circles with GPS sharing, Bluetooth tracker finder, NFC invite.
Health	`/health`	Vitals, daily summaries, screen time, provider connections (e.g. Fitbit), gender-gated cycle tracking.
Wellness	`/wellness`	Breathing sessions and wellbeing items.
Audio & books	`/audio, /audio/[trackId]`	Music, audiobooks and podcasts with chapters, lyrics, ratings, entitlements and subscriptions; RSS import; a process-wide player with queue, shuffle, sleep timer and a draggable mini-player in the app; ebook store with an in-app reader.
Maps & travel	`/map`	Waypoints, trip recorder with animated replay, GPX export, OSRM routing with an offline fallback, one-off location share.

2.6 Sensing and safety

Table 8 — Sensing and safety — routes and capabilities

Capability	Routes	Detail
Smart farm	`/farm, /farm/devices, /farm/rules`	MQTT devices through EMQX and a Node bridge, realtime sensor dashboard, automation rules with cooldowns, alerts, share-as-post.
Smart home	`/home`	Switch toggles, scenes and schedules over the same device layer.
CCTV	`/cameras, /cameras/wall`	RTSP→HLS cameras and KVS WebRTC, zones and PTZ, clips, alerts, group sharing.
SOS	`/map, app`	Reason, phone, note, photo, video and voice, optional Go Live, map danger banner, responder list, SMS + GPS fallback when offline.
Safety check-ins	`app`	Scheduled "I am fine" confirmations with escalation.
Drone radar	`/games/drone-sim, app`	Bluetooth and Wi-Fi based drone detection with RSSI history, quality meter and GPS copy-out.
Wi-Fi mapping	`/admin/wifi`	One row per access point per scan with GPS and RSSI; vendor from OUI; encryption, band, channel congestion, browser and OS analytics; dense-cluster leads explicitly labelled as leads, not evidence.
Threat alerts	`app`	User-reported area alerts with an admin watchlist.

2.7 Platform and administration

Table 9 — Platform — routes and capabilities

Gwave — System Design & Architecture

Capability	Routes	Detail
Admin console	`/admin/*`	Twenty consoles: users, moderation, membership, modules, settings, system, teachers, drivers, games, audio, G-Pay, maps, mines, places, search, storage, AWS cost, Wi-Fi, data.
Developer console	`/dev/*`	Hashed API keys with scopes and rate limits, request logs, HMAC-signed outgoing webhooks, feature flags, OpenAPI 3.1 documentation.
Cost & storage dashboards	`/admin/aws, /admin/storage`	Cost Explorer filtered to `RECORD_TYPE=Usage` so credits cannot mask real consumption; per-table size split into data, index and TOAST; S3 size from CloudWatch daily metrics rather than object listing.
Search analytics	`/admin/search`	Keyword, user, source and result count — zero-result keywords are given equal billing, because that list is the content users wanted and the platform does not have.
Public REST API	`/api/v1/*`	Strains, minerals, posts, sensors and POS products behind scoped keys with per-key rate limits and an OpenAPI document.

Gwave — System Design & Architecture

3 Application architecture

A single Next.js 14 codebase serves server-rendered pages, server actions, JSON APIs for the mobile client, webhooks for external providers and the public REST API. It holds no session state of its own: every request is authenticated from cookies or a bearer token and authorised in the database.

3.1 Layering

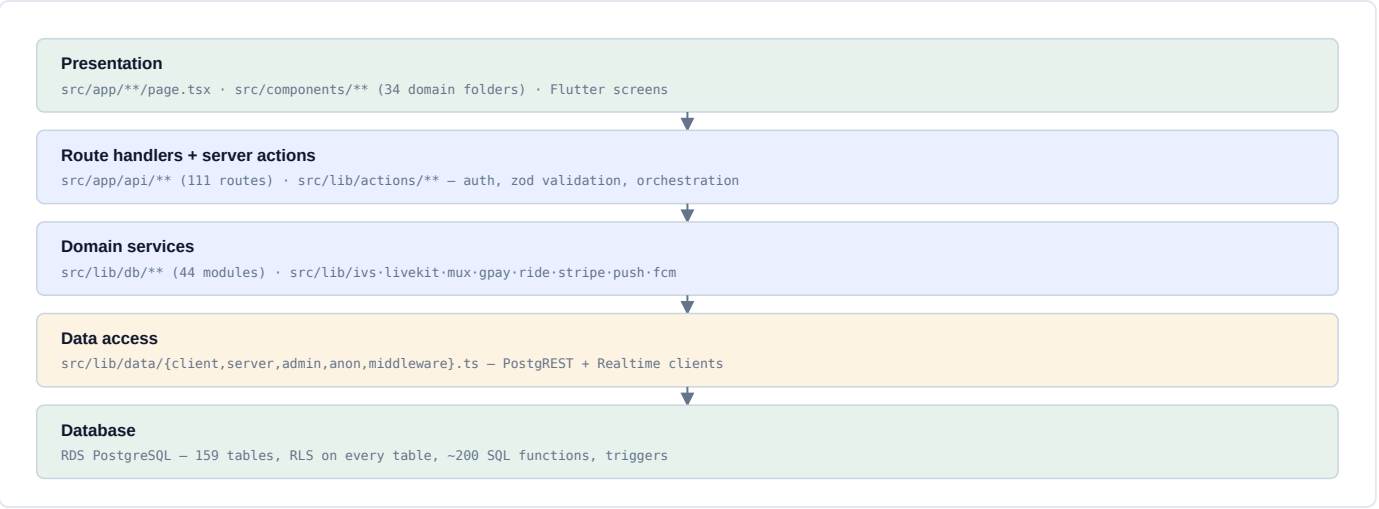

Figure 3 — Application layers. Each layer may call only the one below it; presentation never touches PostgREST directly, and the database is the last line of authorisation rather than the first.

The rule that keeps the codebase navigable is that *authorisation happens twice*: once in the route handler or server action, where a failed check produces a useful error, and once in the database, where row level security produces a correct one. The first exists for the user, the second for correctness — and only the second is trusted.

3.2 Directory map

Table 10 — Repository structure and responsibilities

Path	Responsibility
`src/app/(social|admin|pos|iot|dev|billing|knowledge|tools|auth)`	Route groups; each group owns its layout, navigation and access gate.
`src/app/api/**`	111 route handlers: mobile JSON API, webhooks, media proxies, cron endpoints, public v1 API.
`src/components/**`	34 domain component folders plus `ui/` primitives (shadcn) and `layout/`.
`src/lib/db/**`	44 query modules — one per domain. All PostgREST access is funnelled here.
`src/lib/data/**`	Client factories: browser client, server client, service-role admin client, anonymous client, middleware session refresh.
`src/lib/actions/**`	Server actions: validation, authorisation, orchestration, revalidation.

Gwave — System Design & Architecture

Path	Responsibility
`src/lib/auth/**`	Cognito integration, session cookies, ES256 data-token minting, profile provisioning, phone auth.
`src/lib/{ivs,ivs-realtime,livekit,mux,agora,chime}.ts`	Media provider adapters — the live stack can switch provider per stream.
`src/lib/{gpay,stripe,ride,boost,push,fcm,sms}.ts`	Money, mobility, advertising and notification services.
`supabase/migrations/**`	93 ordered SQL migrations (directory name is historical; they are applied to RDS).
`db/sql/**`	48 patch files applied to production between migrations, each self-contained and idempotent.
`mobile/**`	Flutter application: 38 feature modules, shared core, design system, theme skins.
`deploy/**`	Infrastructure scripts: Caddy, coturn, MediaMTX, LiveKit, KVS master, ECR redeploy, CloudFormation.
`e2e/**, scripts/**`	Playwright smoke and flow suites; RLS audit; k6 load test; seed generators.

3.3 Rendering and data access strategy

- **Server components by default.** Pages fetch through `src/lib/db/*` on the server with the caller's data token, so a page cannot render a row the caller may not read.
- **Client components for interaction only** — composer, player, map, chat, call surface — subscribing to Realtime channels directly.
- **Server actions for writes.** Validation with zod, then a single call into a db module or an RPC, then targeted revalidation.
- **Flat queries on hot paths.** Resource embeds are banned in client-facing queries; related rows are fetched separately and assembled in TypeScript. A stale PostgREST schema cache turns an embed into a 500 and takes a whole feature down silently.
- **Keyset pagination** on the feed and every long list; offset pagination is not used at scale.

> **Why `@supabase/supabase-js` is still a dependency**
> It is a PostgREST and Realtime client, and PostgREST and Realtime are exactly what the self-hosted AWS endpoints speak. The package name is historical; there is no Supabase project, no Supabase dashboard and no hosted service behind any of it. The same applies to every `SUPABASE_*` environment name.

Gwave — System Design & Architecture

4 AWS infrastructure

Production runs entirely inside one AWS account in `ap-southeast-1` (Singapore). There is no managed application platform in the path: the Next.js image runs as a container on an EC2 host behind Caddy, and every other capability is an AWS service the project owns.

4.1 Production topology

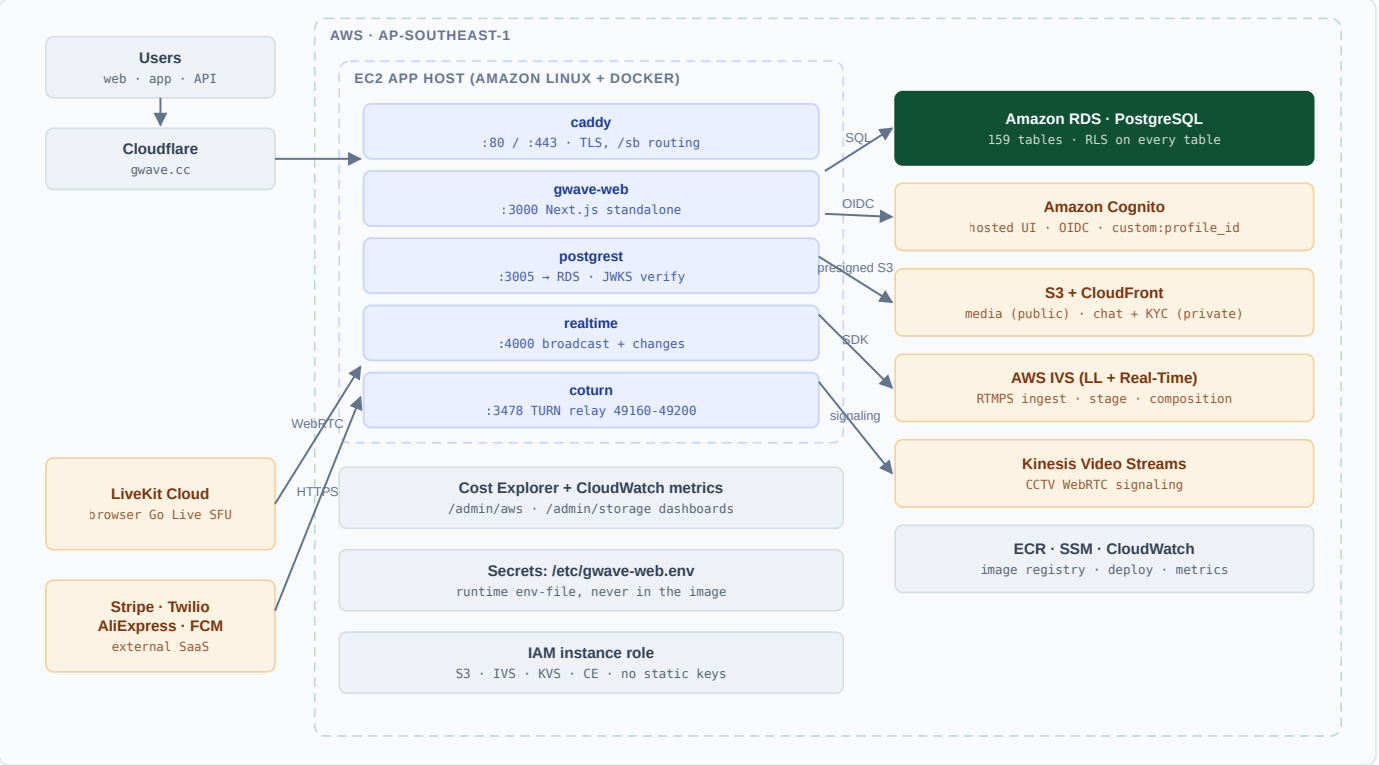

Figure 4 — Production topology. The app host runs five containers; everything else is a managed AWS service reached with the instance role. Only LiveKit Cloud and the payment, SMS and sourcing providers sit outside the account.

4.2 Service inventory

Table 11 — AWS services and what each one carries

Service	Role in Gwave	Configuration notes
EC2	Application host — runs `gwave-web`, `caddy`, `postgrest`, `realtime` and `coturn` as Docker containers.	Managed by SSM (no SSH key in CI). Runtime env lives in `/etc/gwave-web.env`. Redeploys go through `/usr/local/bin/gwave-redeploy`.
RDS PostgreSQL	The single source of truth: 159 tables, row level security on every one, ~200 functions and triggers.	Reached only from the VPC. DDL is applied by an operator with a dockerised `psql`; PostgREST is restarted afterwards to refresh its schema cache.
Amazon Cognito	Identity: hosted UI, OIDC, Google federation, phone auth for the mobile client.	User pool `ap-southeast-1_krSbdHFs9`. `custom:profile_id` is the authoritative link to `profiles.id`.

Gwave — System Design & Architecture

Service	Role in Gwave	Configuration notes
S3 + CloudFront	All file storage: public media, private chat attachments, private KYC and payment slips, IVS recordings.	Writes are browser→S3 with a presigned PUT whose key is derived from the session, never from the client. Private reads are short-lived signed GETs.
ECR	Container registry for the app image.	Two tags per build: `:gwave` (rolling) and `:<sha>` (immutable).
Systems Manager (SSM)	Deployment transport — CI runs the redeploy script on the instance without inbound SSH.	Requires the SSM agent plus `AmazonSSMManagedInstanceCore` on the instance role.
AWS IVS (Low-Latency)	RTMPS ingest for in-app broadcasts and the HLS output every client can play.	Auto-records to S3 through a recording configuration.
AWS IVS (Real-Time)	Browser Go Live stage (WebRTC SFU) plus a composition that restreams the mixed stage into a low-latency channel.	The composition runs even when recording is off — it is how a browser broadcast reaches an HLS URL that phones can play.
Kinesis Video Streams	WebRTC signaling for CCTV cameras reached through the KVS master service.	Deployed from `deploy/kvs-master/` as a systemd unit.
CloudWatch	Container and host metrics; S3 bucket size and object counts for the storage dashboard.	Daily storage metrics are read instead of listing objects — listing the media bucket would be hundreds of thousands of API calls per page load.
Cost Explorer	Powers `/admin/aws`: month-to-date by service, 30-day trend, forecast, credit split.	Billed per request, so the report is cached six hours and refreshing is an explicit button that states its cost.
IAM	Instance role grants S3, IVS, KVS, Cost Explorer and CloudWatch access.	No static access keys on the box. The one place static keys are still required is LiveKit egress, which is why that feature is not yet enabled.

4.3 Non-AWS dependencies

Table 12 — External services

Provider	Used for	Failure behaviour
Cloudflare	DNS, edge caching, DDoS and bandwidth protection in front of the origin.	Origin remains reachable; the in-process rate limiter is the backstop.
LiveKit Cloud	SFU for browser Go Live and co-host rooms.	Absent configuration falls back to a peer-to-peer mesh (practical up to ~6 publishers).
Stripe	Membership checkout and wallet top-up.	Signature-verified webhook; a failed charge surfaces on the receipt instead of becoming a silent support ticket.
Twilio	SMS one-time passwords for G-Pay and phone auth.	OTP start returns a typed error; the flow stops rather than pretending to send.
AliExpress open platform	Dropship product import and refresh.	Import is a manual action; a failed refresh leaves the previous listing intact.
Google Maps / OSRM / Photon	Map tiles, routing, geocoding for ride and travel tools.	Ride search falls back to the rider's own destination history, which is both the cheapest source and the most accurate for Myanmar addresses.

Gwave — System Design & Architecture

Provider	Used for	Failure behaviour
Firebase Cloud Messaging	Planned: background push for calls and messages.	Currently unconfigured — this is the largest known gap.

4.4 Network and port matrix

Table 13 — Inbound ports and their purpose

Port	Protocol	Service	Notes
`80 / 443`	TCP	Caddy	Public entry; automatic TLS; routes `/sb/rest/v1/*` → 127.0.0.1:3005 and `/sb/realtime/v1/*` → 127.0.0.1:4000.
`3000`	TCP (local)	gwave-web	Next.js standalone server, not exposed publicly.
`3001`	TCP (local)	canary	New image is health-probed here before the port-3000 swap.
`3005 / 4000`	TCP (local)	PostgREST / Realtime	Reached only through Caddy under `/sb`.
`3478`	UDP + TCP	coturn	TURN/STUN for messenger calls.
`49160–49200`	UDP	coturn relay	Media relay range. Without it, carrier-NAT calls connect but stay silent.

4.5 Configuration model

Environment variables fall into two classes, and confusing them is the most common deployment mistake in the project.

Class	Where it lives	Consequences
Build-time `NEXT_PUBLIC_*`	Docker build arguments in `deploy.yml`, sourced from repository variables and one secret.	Inlined into the client bundle *and* the Content-Security-Policy. They are public by definition. Changing one requires a rebuild; omitting one silently breaks the feature it powers.
Runtime everything else	`/etc/gwave-web.env` on the EC2 host, read by the redeploy script as `--env-file`.	Never enters the image. An env-only change needs just `sudo gwave-redeploy` — no rebuild, no CI run.

> **Legacy file warning**
> `deploy/gwave.override.env` is legacy and is *not* read by the production redeploy script. Editing it has no effect on the running application; the live file is `/etc/gwave-web.env`.

Gwave — System Design & Architecture

5 Delivery pipeline and environments

Two pipelines exist and only two: one builds and deploys the web application, one builds and publishes the Android APK. Neither has an alternative path, and nothing is deployed by hand.

5.1 Web pipeline

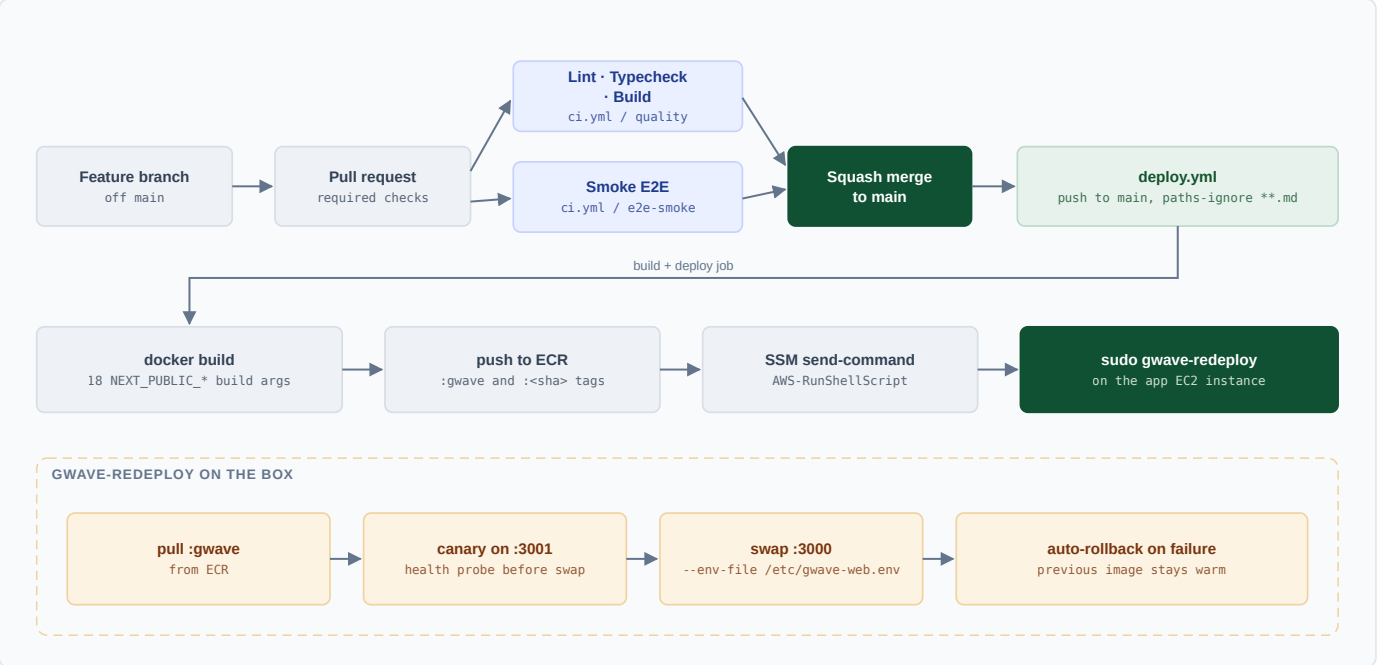

Figure 5 — Web delivery. CI gates the merge; the merge triggers the deploy; the deploy hands off to the same script an operator would run on the box, so there is exactly one deployment procedure.

Table 14 — Web pipeline stages

Stage	Where	What must be true
Quality gate	`ci.yml / quality`	`pnpm lint`, `pnpm typecheck` and `pnpm build` all pass against dummy data-API env.
Smoke E2E	`ci.yml / e2e-smoke`	Playwright checks auth guards, public pages, security headers, API auth and the health probe against the production build.
Merge	GitHub	Squash merge to `main` only after both checks are green.
Image build	`deploy.yml`	18 `NEXT_PUBLIC_*` build args supplied; image tagged `:gwave` and `:<sha>` and pushed to ECR.
Rollout	SSM → EC2	`sudo /usr/local/bin/gwave-redeploy` pulls the new image, canaries it on :3001, swaps :3000 with the runtime env-file, and rolls back automatically on failure.

Deployment concurrency
The deploy workflow uses a concurrency group with `cancel-in-progress: false`: an in-flight production deploy is never interrupted by a newer one. Documentation-only changes are excluded through `paths-ignore`.

Gwave — System Design & Architecture

5.2 Mobile pipeline and branch model

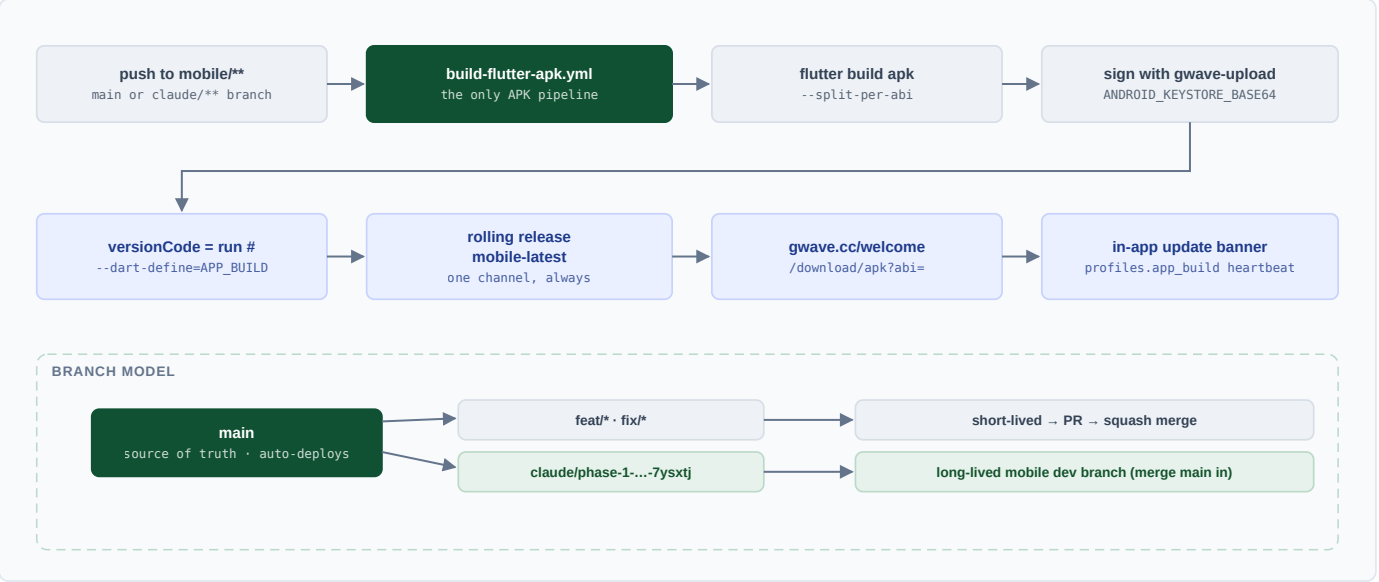

Figure 6 — APK delivery and the branch model. The build number is the CI run number, which makes every screenshot a user sends traceable to an exact build.

- **Signing** uses the `gwave-upload` keystore from `ANDROID_KEYSTORE_BASE64` / `ANDROID_KEYSTORE_PASSWORD`; CI falls back to debug signing with a loud warning if they are absent.
- **Versioning**: `versionCode` is the CI run number, and `--dart-define=APP_BUILD=<run>` powers the in-app update banner. The Settings footer shows `Gwave v1.0.<build>` — always the first thing to ask a user who reports a bug.
- **Distribution** is the rolling `mobile-latest` release; `gwave.cc/welcome` and `/download/apk?abi=…` stream the release assets through the domain so users never touch a third-party host.
- **Telemetry**: the `profiles.app_build` heartbeat column reports each user's installed build, which is how a fleet-wide upgrade is measured.

5.3 Database change process

The application container cannot reach RDS, and neither can CI. This is deliberate: schema changes are the one operation where an automated pipeline would remove the pause in which mistakes are caught.

1. The change is written as a migration in `supabase/migrations/` or a self-contained patch in `db/sql/`.
2. An operator applies it on the EC2 host with a dockerised `psql`, taking the password from `/root/gwaveadmin_newpw.txt` at run time — never hard-coded, never pasted into a chat.
3. After any DDL, `sudo docker restart postgrest` refreshes the schema cache. Skipping this is what turns a new column into a production 500.
4. The corresponding application change is merged only once the schema is in place, so a rollback of the app never leaves the database ahead of it.

> **Validation before production**
> Migrations are validated against a scratch PostgreSQL instance, and `scripts/rls-audit.sql` asserts that every forbidden cross-user operation is denied — the assertions run inside a transaction that is rolled back, so the audit can be run against any database.

Gwave — System Design & Architecture

5.4 Environments

Environment	Data	Purpose
Local development	Own Postgres or a scratch instance; seeds from `supabase/seed/`.	`pnpm dev`. Optional EMQX and IoT bridge through docker compose, with a device simulator.
CI	Dummy data-API endpoints.	Lint, typecheck, production build and smoke E2E — no external service is contacted.
Production	RDS.	gwave.cc. The only environment with real users, real money and real media.

> **There is no staging environment.**
> This is a deliberate trade-off for a small team: the cost of keeping a second full media and payment stack honest exceeded its value. It is mitigated by the canary-and-rollback step in the redeploy script, by feature flags, and by the rule that user-visible state is written before any provider call. It remains the largest structural risk in the delivery model and is recorded as such in §12.

Gwave — System Design & Architecture

6 Security and access control

Authentication is Cognito's job; authorisation is Postgres's. Between them sits a short-lived ES256 token minted by the server, which is the only thing PostgREST and Realtime will accept on a user's behalf.

6.1 Authentication and the data token

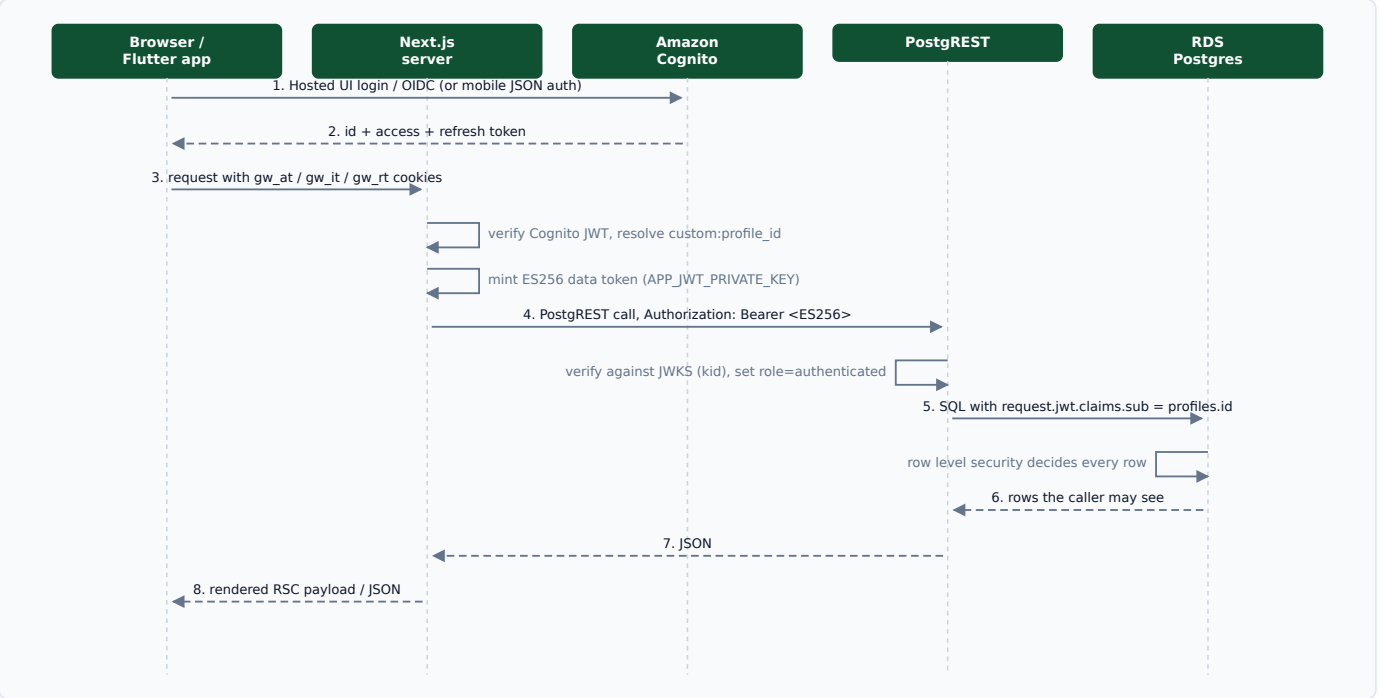

Figure 7 — Request lifecycle. The server never forwards a Cognito token to the data API; it exchanges identity for a narrowly-scoped data token whose subject is the caller's `profiles.id`.

Table 15 — Token and credential types

Credential	Issued by	Trusted by
Cognito id / access / refresh	Cognito hosted UI or the mobile JSON auth routes	The Next.js server only. Stored in the `gw_at` / `gw_it` / `gw_rt` cookies.
ES256 data token	The server, signed with `APP_JWT_PRIVATE_KEY`	PostgREST and Realtime, verified against the JWKS published from `APP_JWT_PUBLIC_JWK` by `kid`. Claims: `sub` = `profiles.id`, `role` = `authenticated`.
Legacy HS256 anon key	Shared secret	PostgREST (as an `oct` key in the same JWKS) and Realtime. Still used for the socket handshake; kept in sync across all three services.
Service-role token	The server, for privileged operations only	Bypasses RLS. Used exclusively from server actions and route handlers via `src/lib/data/admin.ts`.
Public API key	Developer console, stored hashed	`/api/v1/*` only, with per-key scopes (`read:*`, `write:posts`, …) and rate limits.

Gwave — System Design & Architecture

> **The invariant that must always hold**
> The legacy HS256 secret is verified by three independent services — the web container's `SUPABASE_JWT_SECRET`, the Realtime container's `API_JWT_SECRET`, and PostgREST's `oct` key inside `/etc/postgrest/jwks.json`. If any one drifts, every anon-key call fails with `401 JWSInvalidSignature` and the Realtime socket refuses to handshake. The three fingerprints must match; §11 records how to check them.

6.2 Authorisation model

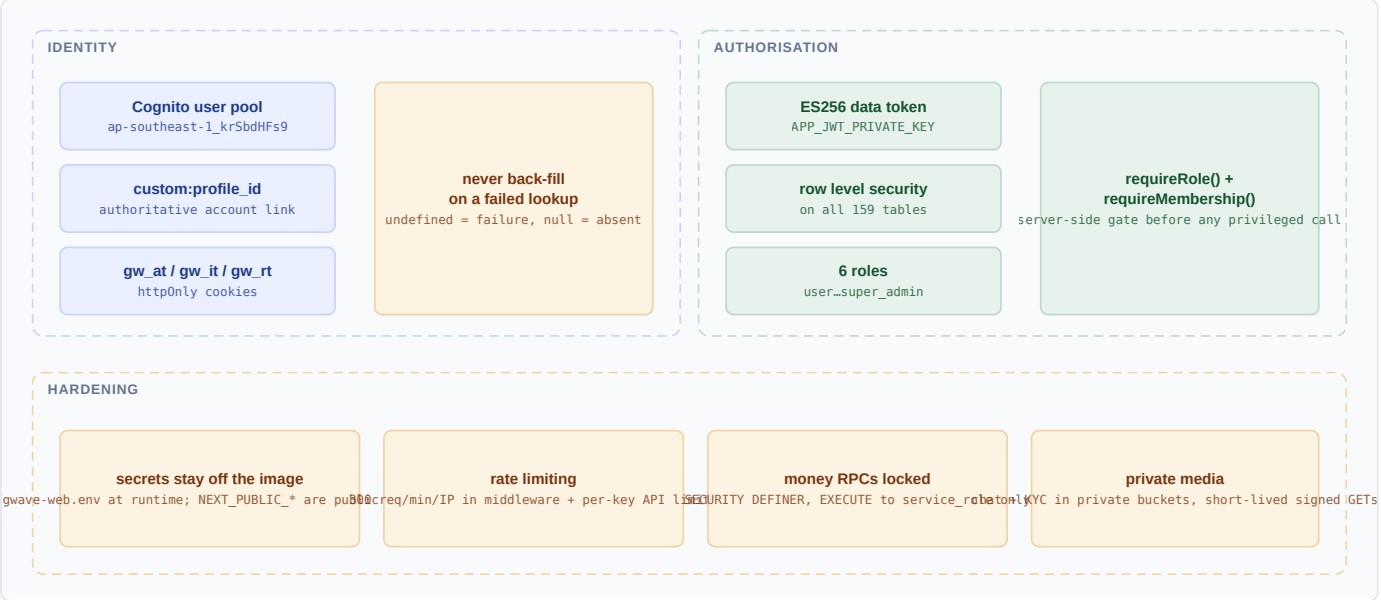

Figure 8 — Access-control model, from identity through authorisation to the hardening measures that assume both will eventually be wrong.

Table 16 — Roles

Role	Grants
`user`	Baseline: own content, friends' content, public content.
`member`	Adds members-only posts, member tools (nutrient mixing, yield estimator) and member badges. Synchronised from an active subscription by `sync_membership_role()`.
`moderator`	Moderation queue, report resolution, content removal.
`developer`	Developer console: API keys, logs, webhooks, feature flags.
`admin`	All admin consoles, user suspension, membership review, settings.
`super_admin`	Everything, including destructive administration and role assignment.

Roles are stored on `profiles.role` and enforced in two places: `requireRole()` and `requireMembership()` in `src/lib/auth.ts` for the application layer, and predicate functions — `is_admin()`, `is_moderator()`, `is_developer()`, `is_member_user()`, `is_suspended()` — inside the RLS policies themselves.

Row level security patterns

- **Ownership**: `owner_id = auth.uid()` — devices, cameras, stores, boosts, webhooks.
- **Relationship**: `are_friends()`, `is_group_member()`, `is_conversation_member()`, `is_family_member()`, `is_store_member()`.
- **Composite visibility**: `can_view_post()` resolves author, visibility level, friendship, group membership, block state and suspension in one predicate — every read path uses the same function, so the feed and a direct permalink can never disagree.

Gwave — System Design & Architecture

- **Age gating**: `is_adult()`, `is_minor()`, `age_band_of()` guard cannabis, dating and market content.
- **Sealed tables**: `wifi_scans` and similar telemetry have *zero* policies — nothing but the service role can read them, which is the correct posture for data collected in the background.

6.3 Data protection

Concern	Control
Secrets at rest	Runtime secrets live only in `/etc/gwave-web.env` and the running containers; they are never baked into the image and never committed. Operational documentation references them by name or by a truncated SHA-256 fingerprint.
Private media	Chat attachments, KYC photos and payment slips live in private buckets; reads go through `/api/media/chat` or short-lived signed GET URLs from `src/lib/storage/signed-read.ts`.
Upload integrity	Object keys are derived from the session user server-side. A client cannot choose where its bytes land.
Money operations	`gpay_protect_columns()` makes `balance` writable only by triggers; transfer, top-up and settlement functions are `SECURITY DEFINER` with `EXECUTE` granted to `service_role` alone.
Stream keys	`live_stream_keys` is a separate host-only table; a viewer of a broadcast can never read the key that produces it.
Rate limiting	Fixed-window 300 requests per minute per IP in middleware (per container), with Cloudflare in front for volumetric protection, plus per-key limits on the public API.
Transport	TLS terminated by Caddy with automatic certificates; security headers asserted by the smoke E2E suite; any `*.vercel.app` host is 308-redirected to `gwave.cc`.
Privacy requests	`consents` and `deletion_requests` record consent state and erasure requests as first-class rows.

6.4 Deliberate privacy decisions

Several features withhold data that would be technically easy to include. These are product decisions, recorded here because they are invisible in the schema.

- **Trip share pages** show the vehicle, plate, driver's first name and live position, and deliberately withhold the rider's identity, both phone numbers, the fare and the payment method. The plate is included because it is what you read out to the police; the rest is not the business of whoever the link gets forwarded to.
- **Only the rider can mint a trip link.** A driver able to publish a passenger's live position would have a stalking tool.
- **Wi-Fi cluster analysis produces leads, not evidence.** Dense access-point clusters are surfaced with an explicit banner — a hotel or a campus looks identical to a scam compound from a scan. The only thing recorded as a judgement is a human admin's flag with an author.
- **Mine and place maps are labelled user-submitted.** Access and hazard notes carry a reported-by-users label, because a map that pretended to know a road was safe would get someone hurt.
- **The police number in a ride is pre-filled, never auto-dialled** — a pocket tap must not call the police.

Gwave — System Design & Architecture

7 Data model

One PostgreSQL database on RDS holds 159 tables joined by 241 foreign keys, with row level security on every table and roughly 200 functions and triggers carrying the business rules that must not be re-implemented per client.

7.1 Shape of the schema

Two structural facts explain most of the schema. First, `profiles` is the hub: 96 of the 241 foreign keys point at it, so almost every table is one join from an identity. Second, counters are *maintained*, not computed — `reaction_count`, `comment_count`, `share_count`, `member_count`, `follower_count` and `clicks_count` are all trigger-updated columns, because the feed cannot afford an aggregate per card.

Table 17 — Schema by domain

Domain	Tables	Anchor tables
Social core	~22	`profiles, posts, comments, reactions, shares, post_media, post_views, stories, reels, follows, friendships, groups, pages, notifications`
Messaging & presence	~12	`conversations, conversation_participants, messages, live_locations, ptt_channels, device_tokens, push_subscriptions`
Live streaming	~11	`live_streams, live_stream_keys, live_chat_messages, live_reactions, live_stream_presence, live_gifts, live_gift_events, live_products, cohost_rooms`
Commerce & wallet	~28	`shop_products, shop_orders, gpay_accounts, gpay_transactions, stores, pos_products, sales, sale_items, shifts, membership_plans, subscriptions, payments, invoices, boosts`
Mobility	~9	`rides, ride_offers, ride_driver_profiles, ride_driver_locations, ride_ledger, ride_driver_balances, ride_vehicle_types, ride_settings`
Learning & knowledge	~20	`strains, minerals, mine_sites, cannabis_places, lesson_progress, member_projects, certificates, lesson_comments, games, game_catalog, typing_scores`
Sensing & safety	~24	`devices, sensor_readings, automation_rules, alerts, scenes, user_cameras, camera_clips, wifi_scans, wifi_networks, drone_detections, sos_alerts, safety_checkins, threat_alerts`
Health, audio & media	~20	`health_vitals, health_daily_summary, health_events, audio_tracks, audio_progress, audio_entitlements, podcast_shows, books, book_progress`
Platform	~13	`api_keys, api_logs, webhooks, webhook_deliveries, feature_flags, site_settings, audit_logs, reports, consents, search_queries`

Gwave — System Design & Architecture

7.2 Social core

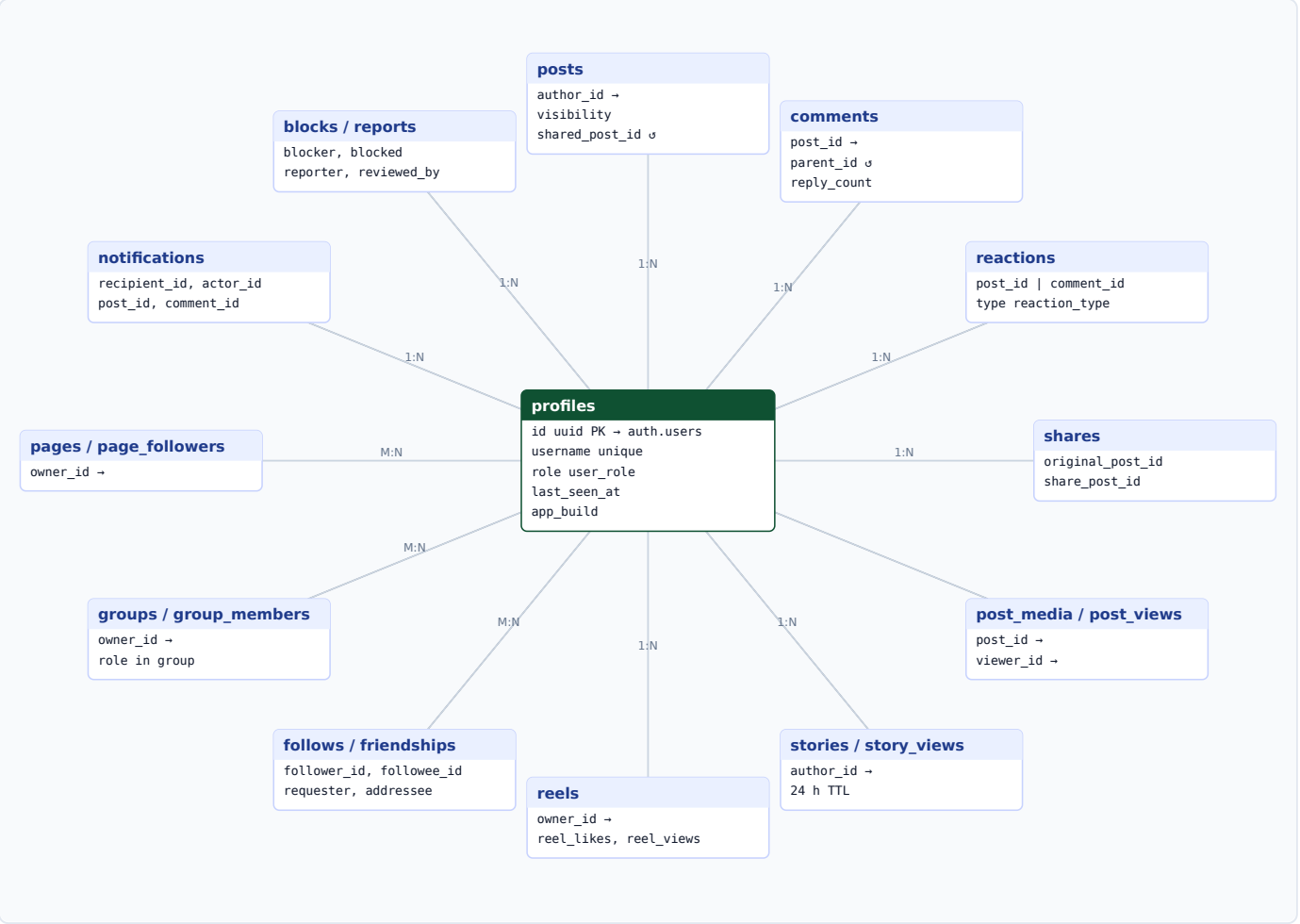

Figure 9 — Social core. Every arm terminates at `profiles`; deletion of a profile cascades through the whole graph, which is why account deletion is a single transaction rather than a job.

Table 18 — Key columns — social core

Table	Columns that carry rules
`profiles`	`id` (PK, references the identity provider), `username` unique with a 3–30 character constraint, `role`, `last_seen_at` (presence: online = seen within two minutes), `app_build` (installed APK build).
`posts`	`visibility` enum, `shared_post_id` self-reference, three trigger-maintained counters, content capped at 10 000 characters.
`comments`	`parent_id` self-reference with `enforce_comment_nesting()` limiting depth; `reply_count` maintained by trigger.
`reactions`	`CHECK ((post_id is null) <> (comment_id is null))` — exactly one target, enforced by the database rather than by convention.
`notifications`	`recipient_id` + `actor_id` + optional `post_id`/`comment_id`; rows are written by triggers, never by clients.

Gwave — System Design & Architecture

7.3 Messaging and presence

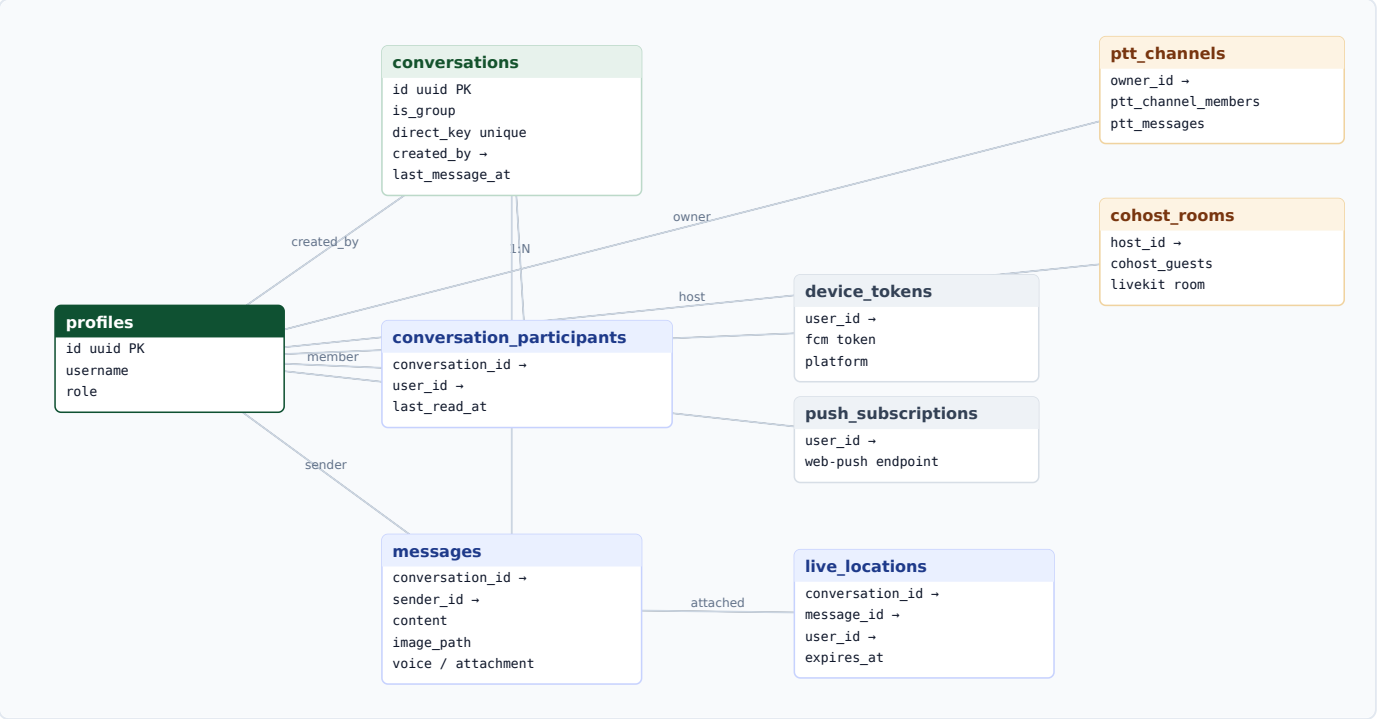

Figure 10 — Messaging. `direct_key` is a unique deterministic key for a one-to-one conversation, which is what makes `get_or_create_direct_conversation()` idempotent under a double tap.

Membership is the authorisation unit: `is_conversation_member()` gates reads and writes on `messages`, and read receipts are stored as `last_read_at` on the participant row rather than per message, which keeps a busy group conversation to one write per reader per visit.

7.4 Live streaming

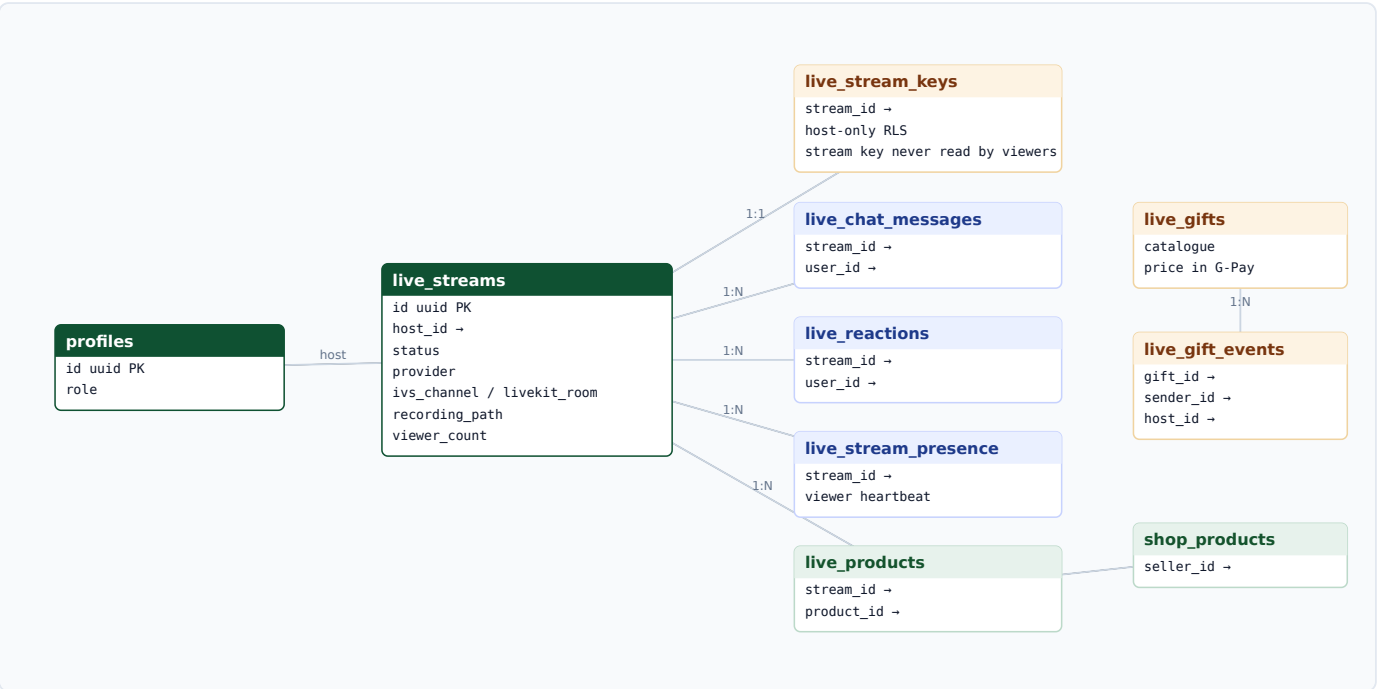

Figure 11 — Live streaming. One `live_streams` row spans every provider path; `provider` and the channel identifier columns decide which media plane a viewer is sent to.

Gwave — System Design & Architecture

`live_streams` is deliberately provider-agnostic. The same row can represent an IVS low-latency channel, an IVS real-time stage restreamed into one, or a LiveKit room, and `recording_path` is filled in later by whichever mechanism finds the finished file first. Column-level lockdown (migration `live_streams_column_lockdown`) prevents a client from writing status or provider fields directly.

7.5 Commerce, wallet and point of sale

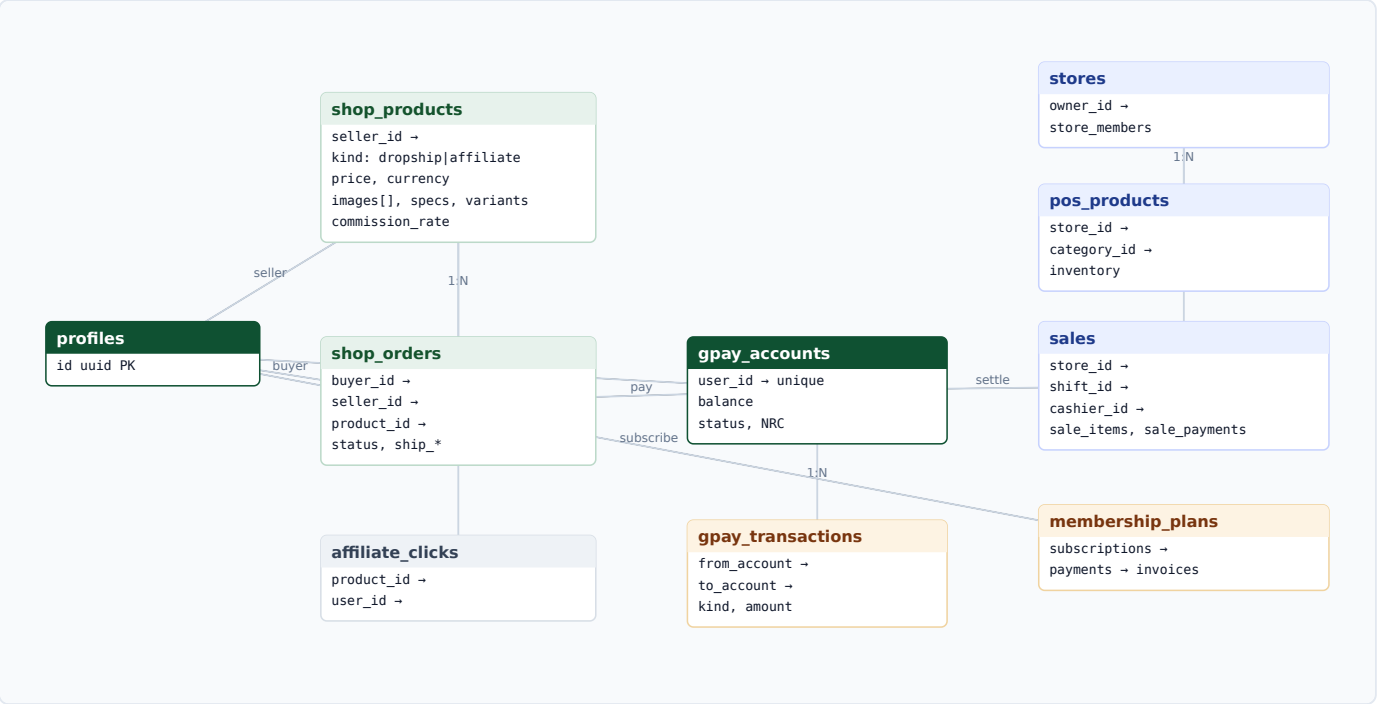

Figure 12 — Commerce. Three separate sales surfaces — marketplace, POS and membership — settle into one wallet, which is why `gpay_accounts` sits in the middle rather than beside them.

Two constraints in `shop_products` encode the product rules in the schema: an affiliate listing must carry an external URL, and a dropship listing must carry a price. The merchant's own list price is deliberately *not* copied into `original_price` — Gwave's price is the merchant's plus a markup, so showing the merchant's "was" beside Gwave's "now" would advertise a discount that does not exist.

Gwave — System Design & Architecture

7.6 Mobility

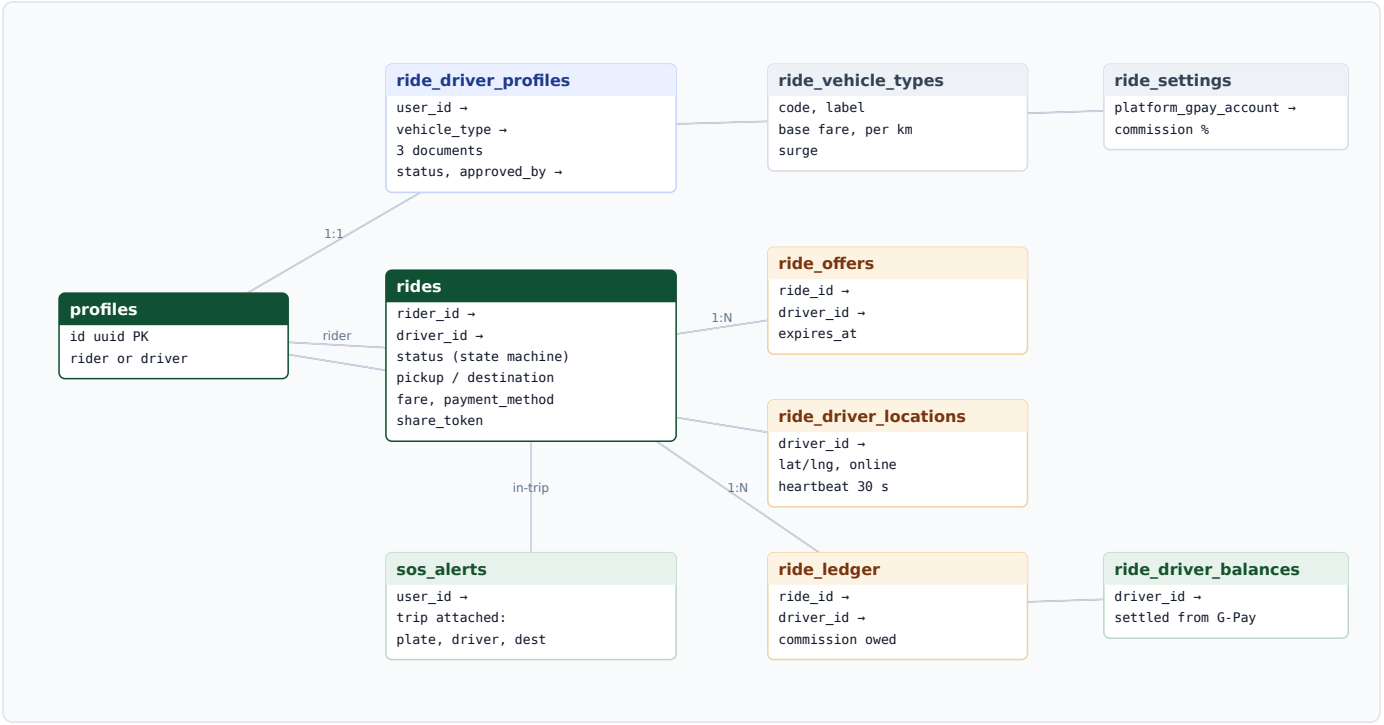

Figure 13 — Ride hailing. The rider and the driver are both `profiles`; what makes someone a driver is an approved `ride_driver_profiles` row, not a second account or a second app.

7.7 Learning and knowledge

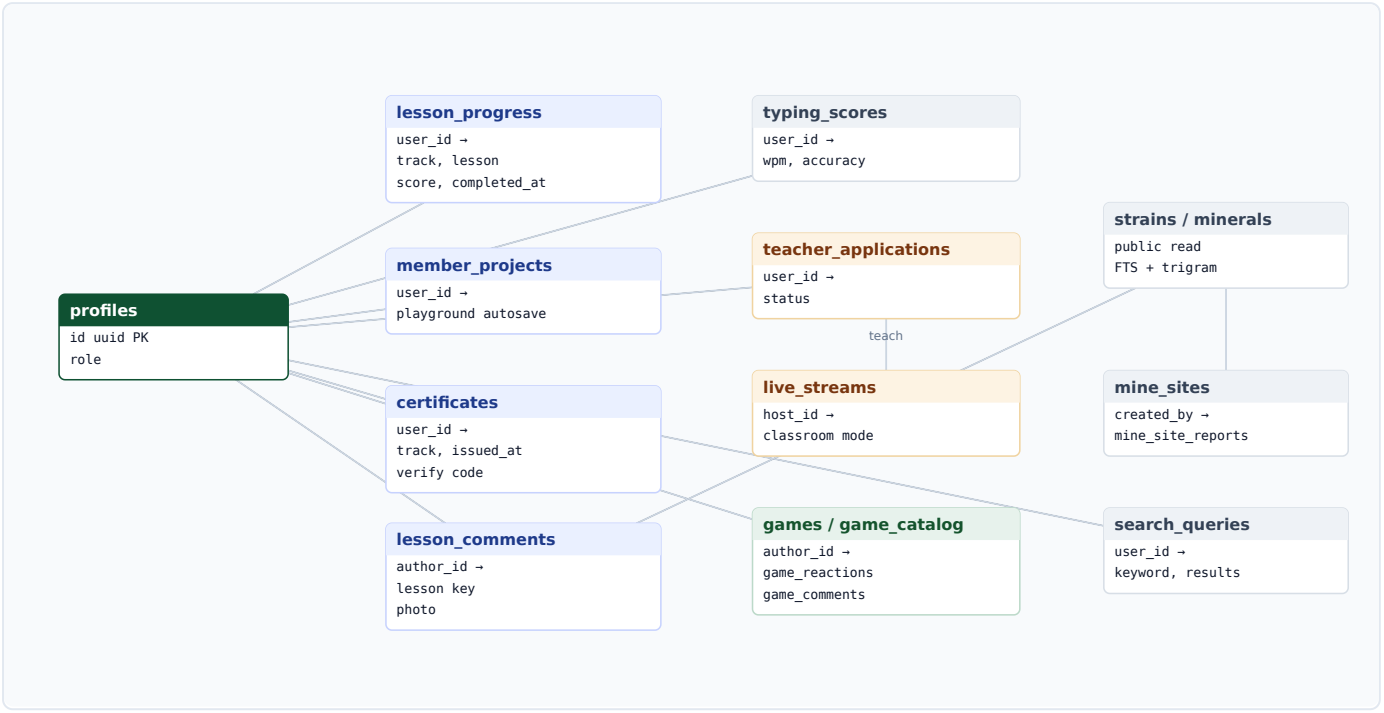

Figure 14 — Learning and knowledge. Progress, projects and certificates are per-user; encyclopedias and community maps are publicly readable and community-writable with moderation.

Gwave — System Design & Architecture

7.8 Sensing, cameras and safety

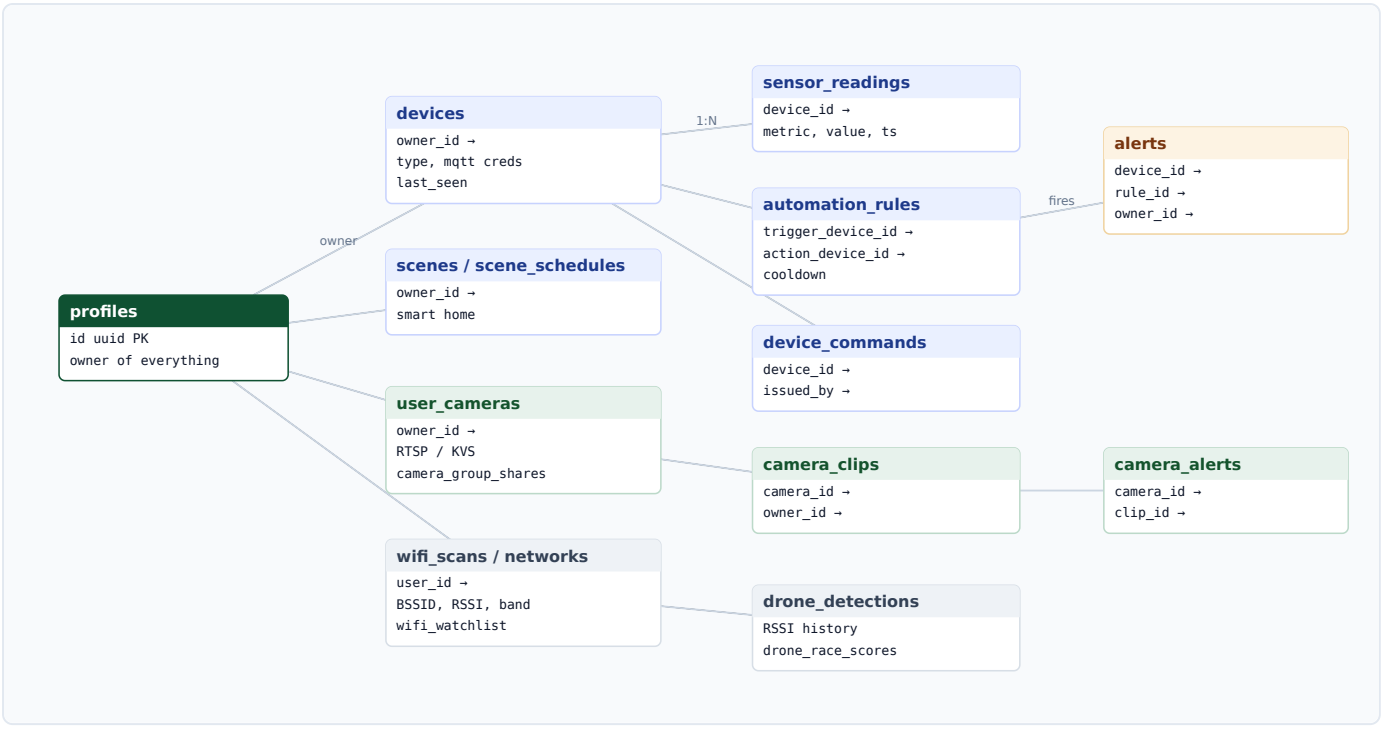

Figure 15 — Sensing. Devices, cameras and scans share one ownership model, so a single RLS predicate covers a farm sensor, a security camera and a Wi-Fi observation.

7.9 Platform services

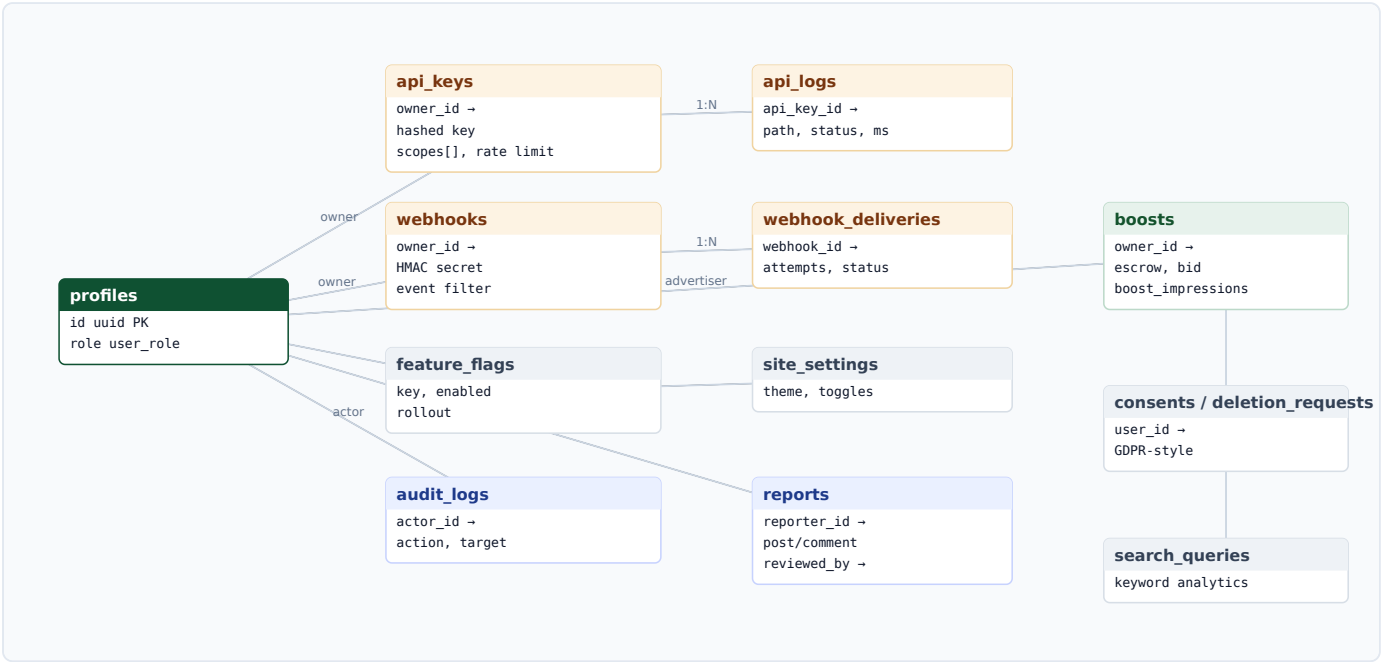

Figure 16 — Platform. API keys, webhooks, flags, audit and moderation — the tables that make the product operable rather than the ones that make it useful.

7.10 Business logic in the database

Roughly 200 functions carry rules that must hold no matter which client is calling. They fall into six groups.

Table 19 — Database function groups

Gwave — System Design & Architecture

Group	Representative functions
Predicates (used inside RLS)	`can_view_post, are_friends, is_conversation_member, is_group_admin, is_store_manager, is_camera_owner, is_family_member, is_adult, is_suspended, is_blocked`
Counter triggers	`bump_reaction_count, bump_comment_count, bump_share_count, bump_view_count, bump_group_member_count, bump_page_follower_count`
Notification triggers	`notify_on_reaction, notify_on_comment, notify_on_share, notify_on_follow, notify_on_friendship, notify_on_alert`
Money (locked down)	`gpay_transfer, gpay_admin_topup, gpay_stripe_topup, gpay_convert, gpay_set_pin, gpay_verify_phone_otp, pos_settle_gpay, ride_settle, ride_driver_settle, buy_audio, buy_book, send_live_gift, withdraw_reel_earnings`
Transactional operations	`create_sale, refund_sale, apply_stock_movement, create_group_with_owner, get_or_create_direct_conversation, ride_accept_offer, ride_guard_transition, place_dropship_order`
Analytics & serving	`get_feed_boosts, record_boost_impression, record_boost_click, boost_daily_stats, creator_stats, creator_leaderboard, learn_leaderboard, review_stats, admin_storage_tables, admin_activity_stats, latest_sensor_readings`

Why so much logic lives in SQL

Two clients (web and Flutter) and a public API all write to the same tables. A rule implemented in TypeScript would hold for one of them. A rule implemented as a constraint, trigger or `SECURITY DEFINER` function holds for all three, including the shipped APK that cannot be patched.

Gwave — System Design & Architecture

8 Functional process designs

Nine flows carry most of the platform's behaviour. Each is described as it actually runs, including the failure that shaped it.

8.1 Publishing a post

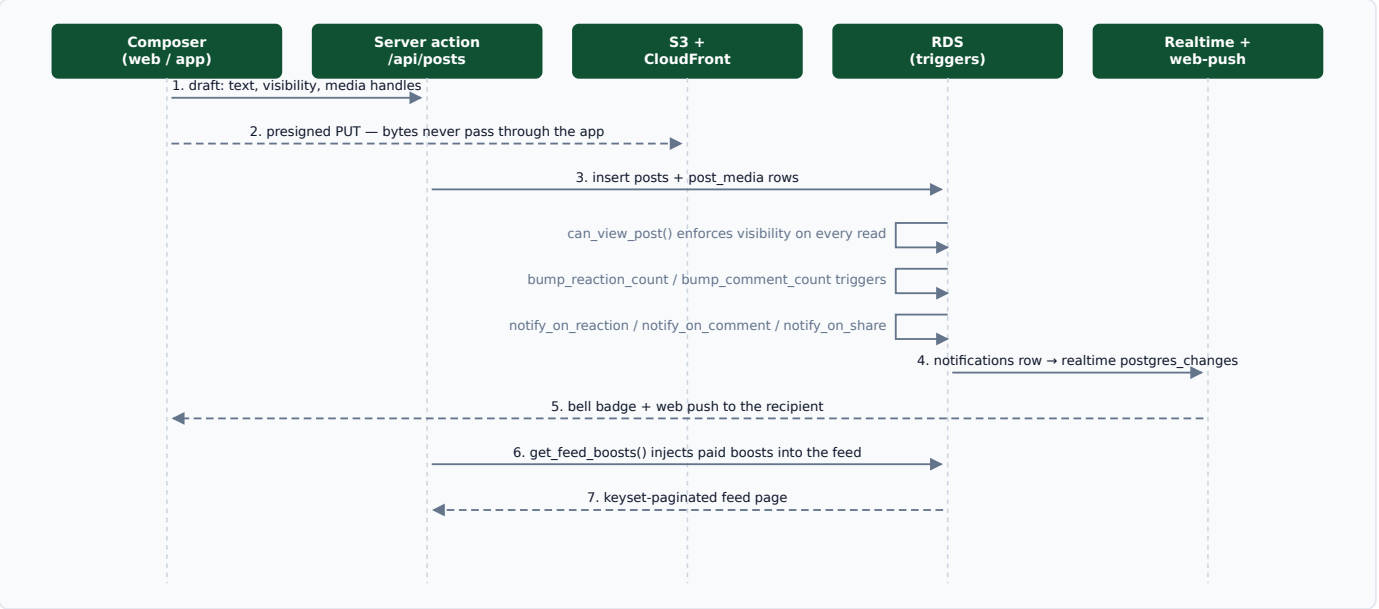

Figure 17 — Post publication and fan-out. Media never passes through the application server; the browser uploads straight to S3 with a presigned PUT whose key the server chose.

1. The composer collects text, visibility and media handles.
2. Each file is uploaded directly to S3 with a presigned PUT from `/api/storage/presign`; the object key is derived from the session user.
3. The server action inserts `posts` and `post_media` in one round trip.
4. Triggers maintain counters and write `notifications` rows for everyone entitled to hear about it.
5. Realtime delivers the notification; web push reaches recipients whose browser is closed but subscribed.
6. Reads go through `can_view_post()`, so the feed, a permalink and a profile grid can never disagree about who may see a post.

Gwave — System Design & Architecture

8.2 Going live

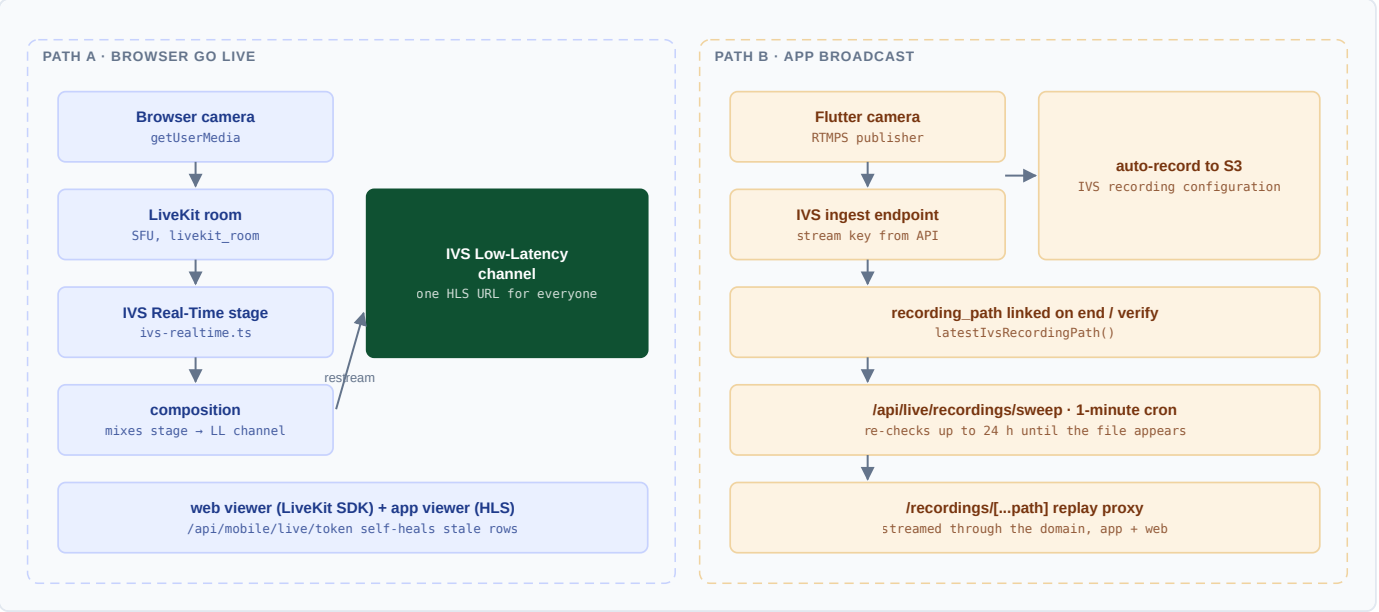

Figure 18 — The two broadcast paths and the single playback contract they both satisfy.

The two paths exist because they solve different problems. A browser can publish WebRTC but not RTMPS; a phone app can publish RTMPS but cannot subscribe to an IVS real-time stage, because doing so requires an IVS Real-Time SDK that Flutter does not have and cannot get. The bridge is the composition: every stage broadcast also gets a low-latency channel, and the composition restreams the mixed stage into it, so the same HLS URL that the app, the web player, the feed card and the live rail already play works for browser broadcasts too.

Three production failures shaped this flow

- **Status written last.**
  `goLive` marked the row live only after awaiting the provider's recording call. The host component fires that action without awaiting it, so a re-render aborted the request mid-flight and Next tore the action down — a host published for nineteen minutes while the row sat at `idle` and every viewer saw nothing. Status is now written first, on its own; provider work is a second update that may fail without costing anything visible.
- **The announcement behind the same abort.**
  The feed post announcing a broadcast was stranded behind the same await, so the broadcast appeared and the post pointing at it never did. Everything a person sees — live status, feed announcement, follower notification — now happens before any AWS call.
- **Replays that finalised too late.**
  IVS finishes writing a recording after the broadcast stops, and how long depends on its length, but the end routes looked the file up the instant the host tapped End and never looked again. Six of ten long broadcasts had a file in the bucket and no path in the row. A one-minute sweeper now re-checks ended broadcasts until the file appears or a day has passed.

Gwave — System Design & Architecture

8.3 Placing and answering a call

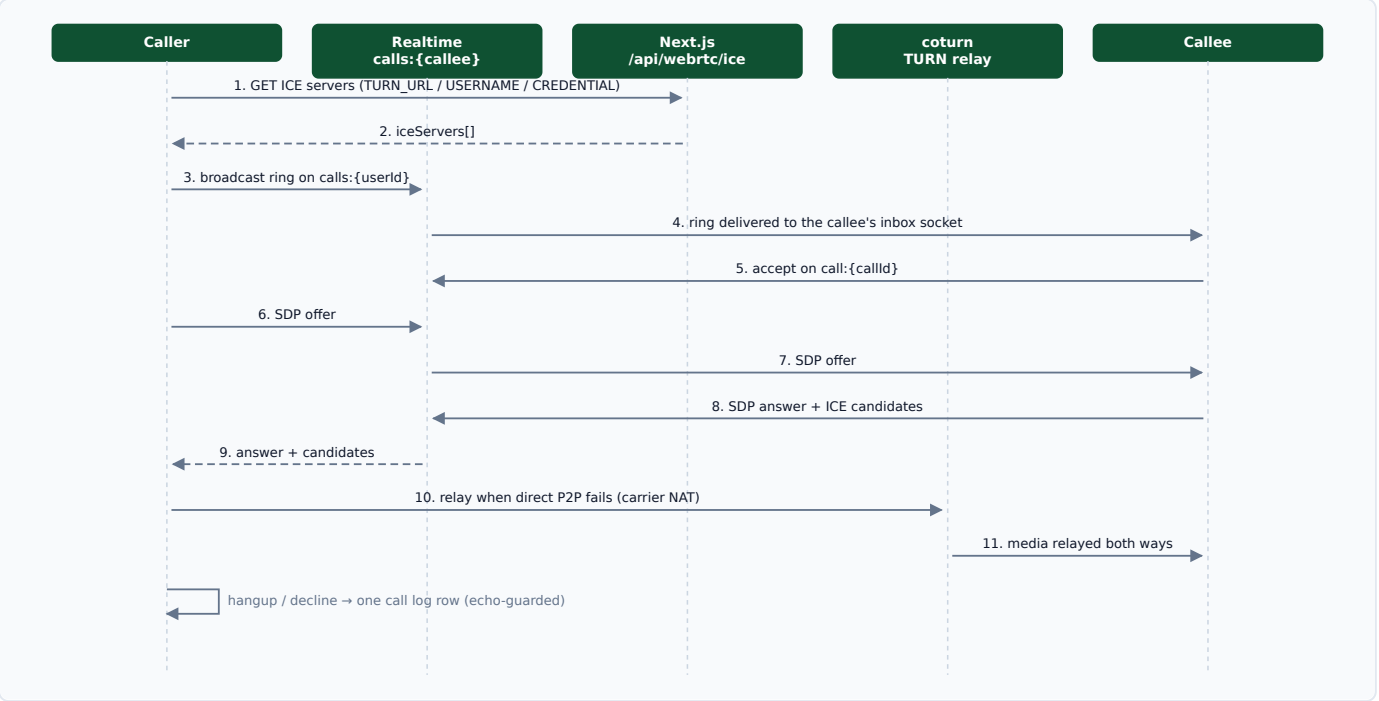

Figure 19 — Call signaling. Signaling runs over Realtime broadcast channels; media is peer-to-peer, with the TURN relay used only when a direct path cannot be established.

- **Ring inbox**: each user subscribes to `calls:{userId}` whenever the app is open; per-call traffic moves to `call:{callId}` once answered.
- **TURN is not optional in practice.** Without a relay, carrier-NAT calls connect and stay silent — the worst possible failure, because both parties believe they are connected.
- **Echo guards.** The server relays a sender's own signal back; `decline` and `hangup` handlers were missing the ownership guard every other handler had, so teardown re-entered and wrote two or three call logs for one call. Fixed with an echo guard, a re-entry lock and a last-logged-call check.
- **Permission dead ends.** A permanently denied camera or microphone permission fails silently on Android; both the chat screen and the call screen now offer a Settings action, because the OS will not show the prompt again.

Gwave — System Design & Architecture

8.4 Dispatching a ride

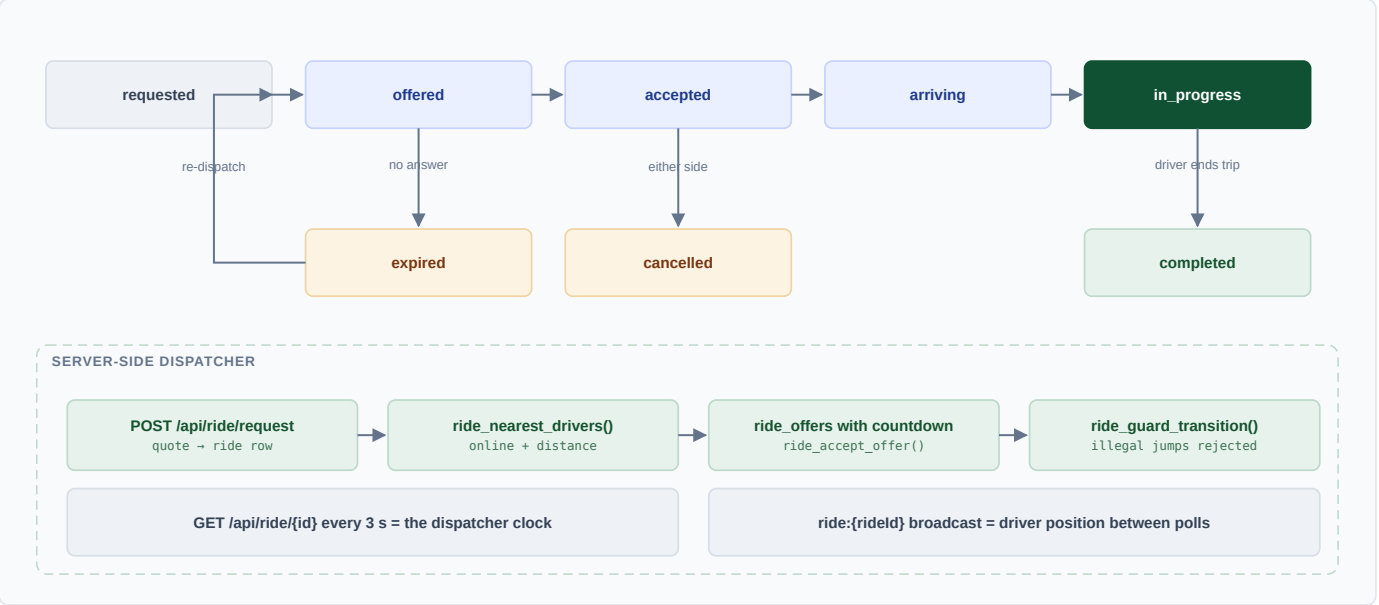

Figure 20 — Ride state machine and the server-side dispatcher. The UI mirrors the server's state rather than keeping one of its own, so the screen cannot reach a state the database disagrees with.

- **The poll is the clock.** `GET /api/ride/{id}` every three seconds also drives dispatch server-side, so no separate scheduler is required for the common case; `/api/ride/dispatch/tick` covers the rest.
- **Position between polls** arrives over `ride:{rideId}` broadcast, so the car glides instead of jumping.
- **Trips survive an app kill.** `/api/ride/active` rejoins on open — without it, a rider reopens Gwave to a fresh booking screen while a driver is on the way to them.
- **Driver presence is process-wide.** The heartbeat was originally a screen-local position stream, so a parked driver (distance filter) or a driver who tapped Back (`dispose()`) stopped beating and was marked offline within three minutes while the switch still read "online". It is now a process-wide service with a 30-second keepalive, stopped only by the switch.
- **Location is a foreground service** with a persistent notification, deliberately not `ACCESS_BACKGROUND_LOCATION`: background location triggers Play Store prominent-disclosure review, the most common reason ride apps get pulled, and offers reach a sleeping screen over push anyway.

Gwave — System Design & Architecture

8.5 Moving money

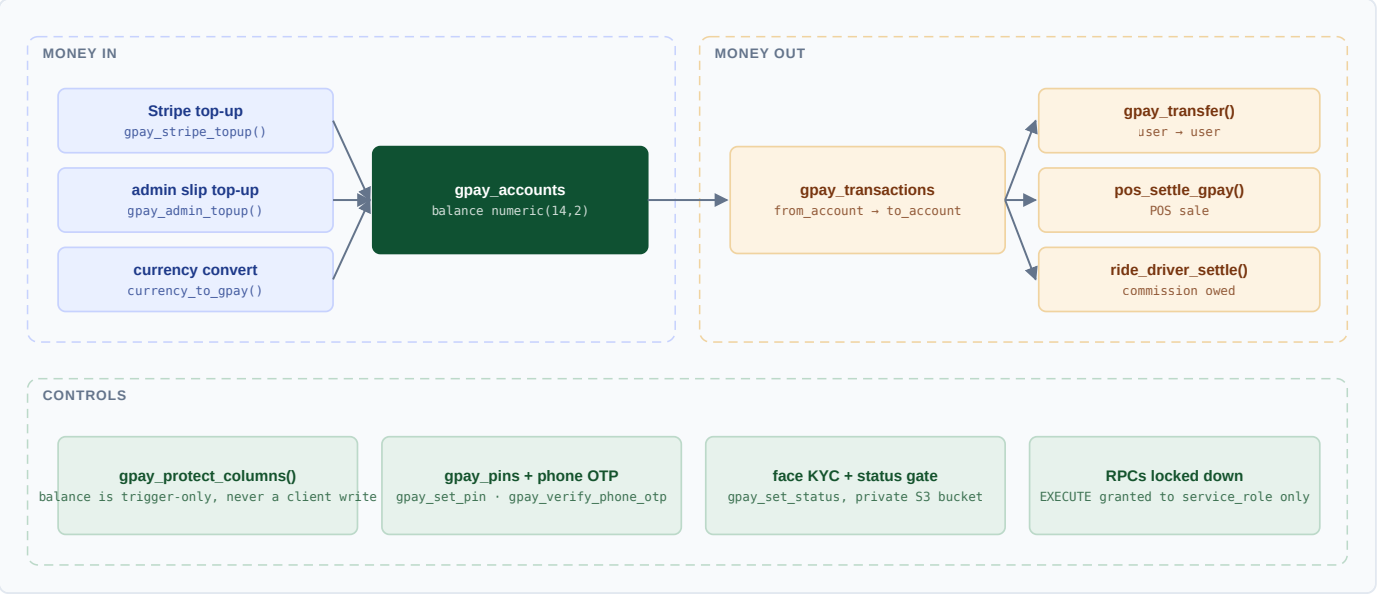

Figure 21 — Wallet flows. Every arrow that changes a balance is a locked-down database function; nothing on the left or right writes `balance` directly.

Cash trips illustrate why the ledger and the money are separate concerns: `ride_driver_settle` had existed and been tested since the schema was written, but nothing called it — so the ledger was correct and the money was not moving. Commission owed is now shown to drivers even at zero, because a driver who only finds out when offers stop assumes the app is broken.

8.6 Selling a dropship product

1. A seller imports a product; `merchantDetail()` writes the full payload — up to 24 images, video, specs, variants, rating, order count, store name and shipping — on *both* import and refresh. The first implementation set them only on import, so a refreshed listing silently reverted to a single photo.
2. The product page renders a real gallery (swipe, arrows, counter, full-screen, video first), a specification table, and the merchant's own numbers shown as *theirs*, separate from Gwave reviews.
3. The seller-only customer pack produces a ready-made sales message, a share-sheet button and a save-every-photo button that fetches blobs — a cross-origin `download` attribute is ignored by browsers, which is why the obvious implementation opens tabs and saves nothing.
4. `place_dropship_order` / `place_dropship_order_gpay` create the order atomically and settle from the wallet.
5. Delivery milestones are stamped by `stamp_order_milestones`, and the buyer receives a push on every status change.

8.7 Serving a boost

`get_feed_boosts()` runs an auction at feed-assembly time: active boosts are ranked by bid weighted against remaining budget and prior delivery, then injected into the feed at fixed positions. Spend is escrowed at creation, `record_boost_impression` and `record_boost_click` account for delivery, and `boost_daily_stats` reports it. Cancelling returns the unspent escrow. The design document lives at `docs/BOOST_ALGORITHM.md`.

Gwave — System Design & Architecture

8.8 Reading a sensor and acting on it

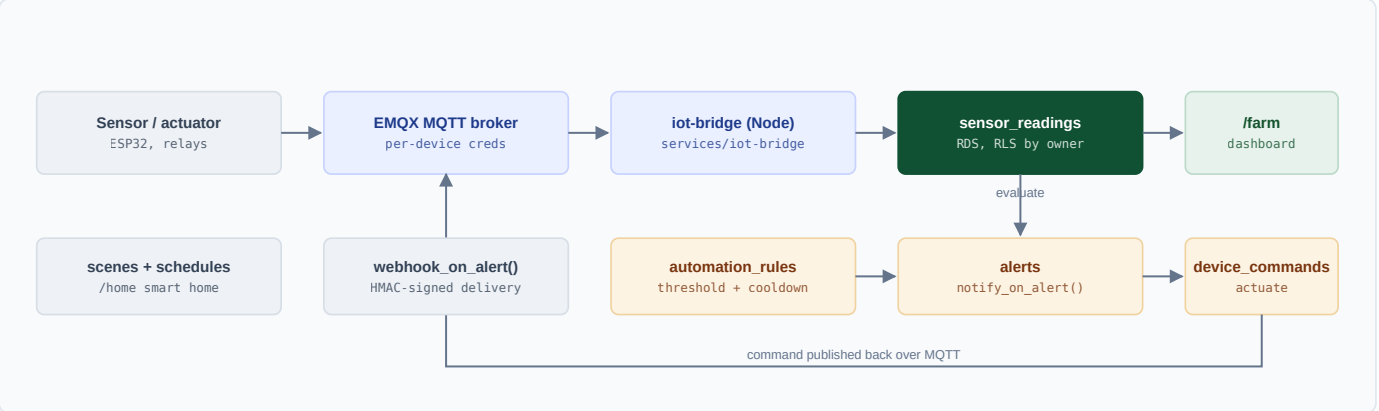

Figure 22 — Smart-farm pipeline. The same device layer serves the farm dashboard and the smart-home screen; an automation rule is a row, not code.

Rules carry a cooldown so a sensor oscillating around a threshold cannot produce an alert storm or cycle a relay. Alerts raise notifications through `notify_on_alert()` and can also fire an HMAC-signed outgoing webhook through `webhook_on_alert()`, which is how the platform integrates with equipment it does not control.

8.9 Raising an SOS

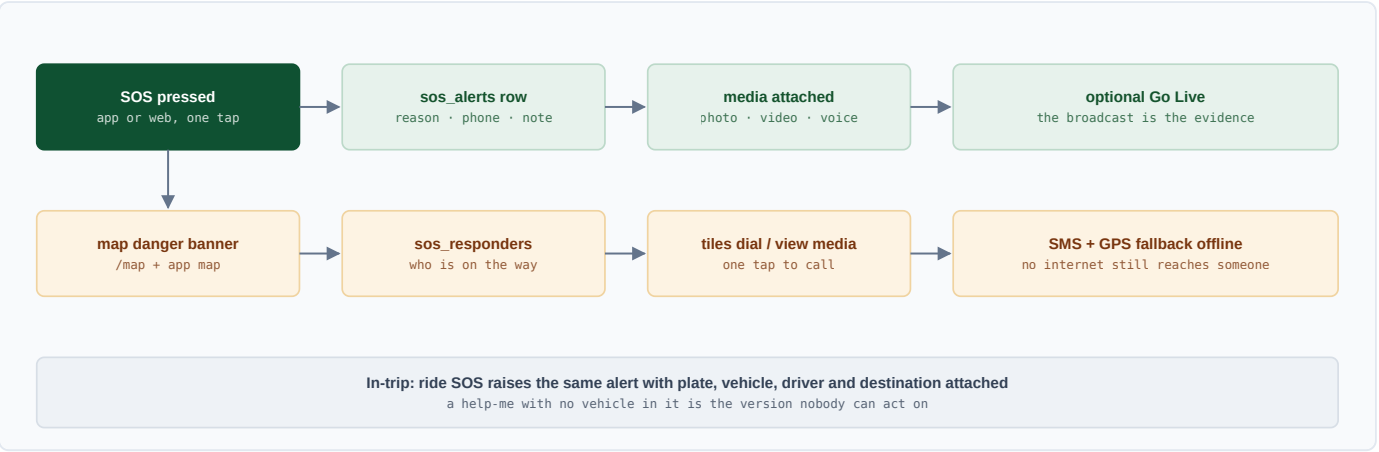

Figure 23 — Safety flow. The alert is one concept: a standalone SOS and an in-trip SOS produce the same row, reach the same map and the same responders.

The SMS + GPS fallback matters more than any other part of the feature: the situations in which someone presses SOS correlate strongly with the situations in which they have no data connection. A help request that requires the internet is a help request that fails when it is needed.

Gwave — System Design & Architecture

9 Mobile application

A native Flutter Android client — not a wrapper — sharing the server, the database and the identity system with the web application, and reaching them through a dedicated JSON API surface.

9.1 Structure

Table 20 — Flutter application structure

Path	Contents
`mobile/lib/core`	HTTP client, auth storage, Realtime socket, theming, localisation (`GwLang` with my/en/th and `tr3()` fallback), video audio focus policy.
`mobile/lib/features`	38 feature modules: feed, messenger, live, ride, gpay, shop, learn, health, audio, books, family, farm, cctv, drone, map, market, dating, reels, stories, talk, pos, jobs, games, tools, settings, shell and more.
`mobile/lib/widgets`	Shared design-system widgets, signal meters, media tiles, mini player.

9.2 Platform capabilities the app adds

- **Process-wide audio.** One `GwAudio` service owns playback; queue, shuffle, repeat, speed, sleep timer and position saving live on the service, so they survive leaving the player screen. A muted feed video no longer stops the music: previews use `mixWithOthers` and only an unmuted video takes audio focus. What is playing is reachable from every tab through a draggable floating bubble that costs no layout and is hidden over Reels.
- **Offline peer-to-peer chat** over Nearby Connections (Bluetooth / Wi-Fi Direct) with GPS sharing, and explicit Burmese guidance when Play Services Nearby is unavailable.
- **Sensing**: Wi-Fi scanning with frequency, channel, band and raw capabilities; Bluetooth drone detection with RSSI history; NFC read and write.
- **Travel tools**: long-press waypoints stored on device, a trip recorder with animated replay, GPX export, in-app OSRM routing with a straight-line fallback when offline.
- **Theme skins**: four complete visual identities (Gwave, Sky, Liberty, Tactical) switchable at runtime.
- **Live viewing** as a one-page vertical swipe, with muted autoplay previews in the feed and live lists.

9.3 Mobile API surface

The app never speaks to PostgREST for privileged operations; it uses a dedicated JSON API under `/api/mobile/*` — 30 routes covering auth (login, register, Google, phone start/verify, refresh), live (create, start, end, token, verify), calls (signal, notify), wallet (register, top-up), learning, audio publishing, dating, marketplace, uploads, push registration, Wi-Fi and drone observation, PTT, text-to-speech and configuration.

> **Why a separate surface**
> A mobile client cannot hold a browser session cookie, cannot run a server action, and cannot be redeployed to fix a query. Giving it explicit endpoints means the contract is versioned in one place and a shipped APK keeps working when the web UI changes underneath it.

Gwave — System Design & Architecture

10 Public API and integrations

10.1 Public REST API

Table 21 — Public API endpoints

Endpoint	Scope	Returns
`GET /api/v1/strains`	`read:strains`	Strain encyclopedia entries with search and pagination.
`GET /api/v1/minerals`	`read:minerals`	Mineral encyclopedia entries.
`GET/POST /api/v1/posts`	`read:posts / write:posts`	Public posts; creation on behalf of the key owner.
`GET /api/v1/sensors`	`read:sensors`	Latest readings for devices owned by the key owner.
`GET /api/v1/pos/products`	`read:pos`	Products for stores the key owner can manage.
`GET /api/v1/openapi.json`	—	OpenAPI 3.1 description of the whole surface.

Keys are stored hashed, carry an explicit scope list and a rate limit, and every request is written to `api_logs` with path, status and duration. Authorisation is checked in `src/lib/api-auth.ts` against `src/lib/api-scopes.ts` before any data access.

10.2 Inbound webhooks

Endpoint	Purpose and verification
`/api/webhooks/stripe`	Membership and top-up settlement. Signature-verified against `STRIPE_WEBHOOK_SECRET`.
`/api/live/ivs-stream-webhook`	IVS channel state changes (stream start/stop).
`/api/live/ivs-recording-webhook`	Recording finalisation events from EventBridge; idempotent alongside the sweeper.
`/api/live/livekit-webhook`	LiveKit room and participant events.
`/api/live/webhook`	Legacy Mux path, retained for the RTMP/OBS fallback.
`/api/health/[provider]/callback`	OAuth callbacks for connected health providers.

10.3 Outgoing webhooks

Developers register endpoints with an event filter; deliveries are queued by `enqueue_webhook()`, signed with an HMAC of the payload, and recorded in `webhook_deliveries` with attempt counts and status. Triggers `webhook_on_post`, `webhook_on_sale` and `webhook_on_alert` are the current event sources.

10.4 Scheduled work

Job	Cadence	Purpose
`/api/live/recordings/sweep`	1 minute	Link replay paths that finalise after the host has left; end stale broadcasts; start compositions for live stages missing one. `?hours=` up to 720 for a one-off catch-up.
`/api/ride/dispatch/tick`	On demand	Advance dispatch when no rider poll is driving it.
`/api/heartbeat`	Client-driven	Presence: `profiles.last_seen_at`, 60-second timer in the app.

Gwave — System Design & Architecture

Job	Cadence	Purpose
Story cleanup	Daily	Expire stories past 24 hours.

Sweep endpoints authenticate with `LIVE_SWEEP_SECRET`, falling back to `RIDE_DISPATCH_SECRET`, so adding a scheduled job required no new secret.

Gwave — System Design & Architecture

11 Operations and observability

11.1 Health and dashboards

Surface	What it answers
`/api/health`	Liveness probe used by the canary swap and by uptime monitoring.
`/api/diag, /api/mobile/diag`	Configuration and connectivity diagnostics for support triage.
`/admin/system`	Container and service status.
`/admin/data`	Every module's totals, per-table row counts with seven-day growth, log-scale toggle.
`/admin/storage`	Per-table size split into data, index and TOAST; S3 bucket size and object count from CloudWatch.
`/admin/aws`	Cost by service month-to-date, last-month comparison, 30-day trend, forecast, credit split, and for each service a note on what Gwave runs on it and the one lever that reduces it.

A dashboard that hides the truth is worse than none
Grouping Cost Explorer by service without a record-type filter nets credits into each service. On this account that made EC2 read $0.01 against $149 of actual usage, and the whole bill read $52 when real consumption was $228.50/month against $180/month of credits. The service table and the daily trend are now filtered to `RECORD_TYPE=Usage`, and the page leads with real usage and warns about the gap — because the day the credits run out, the invoice jumps 4.5× with no warning.

11.2 Symptom-to-cause reference

Table 22 — Production triage

Symptom	Likely cause
`401 JWSInvalidSignature` on `/sb/rest`	PostgREST's JWKS lacks the current ES256 public key (`kid` mismatch), or the legacy `oct` key no longer equals the current secret.
Realtime "alg / signature" errors	`API_JWT_SECRET` on the realtime container has drifted from the web container's secret.
Upload fails	`/api/storage/presign` rejected the request, or the EC2 instance role lost S3 write permission.
`permission denied for table X`	Missing GRANT to `anon`, `authenticated` or `service_role` after a new table was created.
`new row violates row-level security`	Missing INSERT policy, or a privileged write attempted without the service role.
A feature 500s right after a schema change	PostgREST's schema cache is stale — `sudo docker restart postgrest`.
Images blank immediately after a deploy	Stale JS chunks (deploy skew). A hard reload fixes the client.
Calls connect but stay silent	TURN relay unreachable — check coturn and the UDP relay range in the security group.

Gwave — System Design & Architecture

11.3 Testing and audits

Check	Coverage
Smoke E2E	Auth guards, public pages, security headers, API authentication, health probe — runs in CI against the production build.
Full flows	Post, react, comment, search, POS sale, API keys. Requires a seeded database: `E2E_FULL=1 pnpm e2e`.
RLS audit	`scripts/rls-audit.sql` asserts every forbidden cross-user operation is denied, then rolls back.
Load test	`scripts/load-test-feed.js` (k6) — feed keyset pagination, target p95 under 800 ms at 50 virtual users.
Merge hygiene	`git grep '^<<<<<<< '` after every merge; conflict markers must never reach a commit.

11.4 Backup and recovery

- **Database**: RDS automated backups and snapshots are the recovery point for all application data.
- **Media**: S3 holds every uploaded object and every IVS recording; the application stores paths, never bytes.
- **Application**: every deploy is an immutable ECR image tagged with its commit SHA, so any prior version can be re-run.
- **Ad-hoc containers**: `realtime` and `postgrest` run as plain `docker run`, so their configuration lives only in the running containers. `deploy/realtime-run.sh` recreates the realtime container from a captured env file. *PostgREST is not yet scripted* — capturing its environment and its JWKS bind mount is an open operational task recorded in §12.

Gwave — System Design & Architecture

12 Roadmap, risks and known gaps

12.1 Open gaps

Table 23 — Known gaps, in priority order

Gap	Impact	What it needs
FCM background push	**High**	A Firebase project. Calls and messages do not ring when the app is closed; web push covers open browsers only. This is the difference between a messenger and a messenger people rely on.
LiveKit egress recording	**Medium**	`LIVEKIT_EGRESS_S3_*` plus static IAM keys in `/etc/gwave-web.env`, so browser Go Live sessions get replays the way app broadcasts already do.
PostgREST container not reproducible	**Medium**	Capture its environment and JWKS mount into a launcher script mirroring `deploy/realtime-run.sh`. If the box is lost today, that configuration is lost with it.
No staging environment	**Medium**	Accepted trade-off; mitigated by canary, rollback and feature flags. Revisit when a second engineer joins.
Native iOS	**Medium**	Apple Developer Program enrolment. The PWA install guide on `/welcome` is the interim path.
Chime SDK messenger calls	**Low**	Planned third IVS/AWS phase; the current WebRTC + coturn path works and this would mainly reduce operational surface.
Cost headroom	**Watch**	Real consumption of roughly $228.50/month against $180/month of credits. The lever list per service is on `/admin/aws`.

12.2 Structural risks

- **Single application host.** The web container, PostgREST, Realtime and coturn share one EC2 instance. It is inexpensive and simple, and it is a single point of failure; the ECS Fargate path documented in `docs/AWS_DEPLOY.md` is the prepared escape route.
- **Schema breadth.** 159 tables in one database is a large surface for one team. It is held together by uniform conventions — ownership columns, RLS on everything, counters by trigger — and those conventions are the thing to defend when adding the 160th.
- **Provider concentration in the live stack.** Three media providers are supported, which is resilience, but the composition path that makes browser broadcasts playable on phones is IVS-specific.
- **Operational knowledge.** Several procedures exist only as runbook text. The mitigation is that every one of them is written down in `docs/` rather than held by a person.

12.3 Sequenced next steps

1. Stand up the Firebase project and wire FCM end to end — highest user-visible return of anything on this list.
2. Configure LiveKit egress so both broadcast paths produce replays.
3. Script the PostgREST container, closing the last unreproducible piece of infrastructure.
4. Add per-service cost alarms against the credit runway.
5. Enrol in the Apple Developer Program and build the iOS target from the existing Flutter codebase.

Gwave — System Design & Architecture

A Appendix A — Web route inventory

Every rendered page in the application, 147 in total, grouped by App Router route group. Route groups do not appear in URLs; they scope layout and access.

Administration consoles (20 pages)

`/admin`
`/admin/audio`
`/admin/aws`
`/admin/data`
`/admin/drivers`
`/admin/games`
`/admin/gpay`
`/admin/map`
`/admin/membership`
`/admin/mines`
`/admin/moderation`
`/admin/modules`
`/admin/places`
`/admin/search`
`/admin/settings`
`/admin/storage`
`/admin/system`
`/admin/teachers`
`/admin/users`
`/admin/wifi`

Authentication (4 pages)

`/forgot-password`
`/login`
`/register`
`/reset-password`

Membership and billing (2 pages)

`/membership`
`/membership/promptpay/[plan]`

Developer console (6 pages)

`/dev`
`/dev/api-docs`
`/dev/deploy`
`/dev/flags`
`/dev/logs`
`/dev/webhooks`

Smart farm and smart home (4 pages)

`/farm`
`/farm/devices`
`/farm/rules`
`/home`

Knowledge bases and markets (8 pages)

`/metals`
`/minerals`
`/minerals/[slug]`
`/minerals/mines`
`/strains`
`/strains/[slug]`
`/strains/market`
`/strains/places`

Point of sale (8 pages)

`/pos`
`/pos/inventory`
`/pos/receipts`
`/pos/receipts/[id]`
`/pos/reports`
`/pos/sell`
`/pos/shifts`
`/pos/staff`

Social, media, commerce and life services (80 pages)

`/audio`
`/audio/[trackId]`
`/boost`
`/boost/new`
`/cameras`
`/cameras/[id]`
`/cameras/wall`
`/dashboard`
`/family`
`/family/trackers`
`/feed`
`/finance`

Gwave — System Design & Architecture

/friends
/games
/games/[id]
/games/creators
/games/drone-sim
/games/edu/[slug]
/games/grow
/games/submit
/gpay
/groups
/groups/[slug]
/health
/help
/jobs
/jobs/[id]
/jobs/[id]/edit
/jobs/applications
/jobs/manage
/jobs/manage/[id]
/jobs/new
/leaderboard
/learn
/learn/[track]
/learn/[track]/[lesson]
/learn/certificate/[id]
/learn/game
/learn/languages
/learn/languages/[lang]
/learn/languages/[lang]/[unit]
/learn/languages/typing
/learn/leaderboard
/learn/live
/learn/live/new
/learn/playground
/learn/teach
/live
/live/[id]
/live/cohost
/live/cohost/[code]
/live/dashboard
/live/new
/map
/meet
/meet/[room]
/notifications
/p/[id]
/pages
/pages/[slug]
/profile
/reels
/reels/analytics
/restricted
/search
/settings
/settings/import
/shop
/shop/[id]
/shop/dashboard
/shop/guide
/shop/guide/affiliate
/shop/guide/dropship
/shop/orders
/shop/sales
/shop/sell
/talk
/talk/[id]
/u/[username]
/wellness

Grower tools (9 pages)

/tools
/tools/converters
/tools/currency
/tools/ec-converter
/tools/nutrient-calculator
/tools/profit
/tools/qr
/tools/vpd
/tools/yield-estimator

Top level (no route group) (6 pages)

/
/messages
/onboarding
/ride/track/[token]
/watch/[token]
/welcome

Gwave — System Design & Architecture

B Appendix B — API route inventory

All 111 route handlers under `src/app/api`.

Mobile client API (38)

- `/mobile/audio/import-rss`
- `/mobile/audio/publish`
- `/mobile/auth/google`
- `/mobile/auth/login`
- `/mobile/auth/phone/start`
- `/mobile/auth/phone/verify`
- `/mobile/auth/refresh`
- `/mobile/auth/register`
- `/mobile/books/publish`
- `/mobile/call/notify`
- `/mobile/call/signal`
- `/mobile/config`
- `/mobile/dating`
- `/mobile/dating/candidates`
- `/mobile/dating/matches`
- `/mobile/dating/swipe`
- `/mobile/diag`
- `/mobile/drone/nearby`
- `/mobile/gpay/register`
- `/mobile/gpay/topup`
- `/mobile/learn/languages`
- `/mobile/learn/lesson`
- `/mobile/learn/tracks`
- `/mobile/live/create`
- `/mobile/live/end`
- `/mobile/live/start`
- `/mobile/live/token`
- `/mobile/live/verify`
- `/mobile/market`
- `/mobile/market/status`
- `/mobile/ptt/create`
- `/mobile/ptt/join`
- `/mobile/push/register`
- `/mobile/subject-comments`
- `/mobile/tts`
- `/mobile/upload`
- `/mobile/wifi/nearby`
- `/mobile/wifi/observe`

Live streaming (8)

- `/live/[id]/delete`
- `/live/[id]/end`
- `/live/create`
- `/live/ivs-recording-webhook`
- `/live/ivs-stream-webhook`
- `/live/livekit-webhook`
- `/live/recordings/sweep`
- `/live/webhook`

Ride hailing (16)

- `/ride/[id]`
- `/ride/[id]/cancel`
- `/ride/[id]/rate`
- `/ride/[id]/share`
- `/ride/[id]/status`
- `/ride/active`
- `/ride/admin/drivers`
- `/ride/dispatch/tick`
- `/ride/driver/apply`
- `/ride/driver/heartbeat`
- `/ride/driver/settle`
- `/ride/offers/respond`
- `/ride/places`
- `/ride/quote`
- `/ride/request`
- `/ride/track/[token]`

Administration (4)

- `/admin/audio`
- `/admin/aws-cost`
- `/admin/members.csv`
- `/admin/wifi-watchlist`

Public REST API (6)

- `/v1/minerals`
- `/v1/openapi.json`

Gwave — System Design & Architecture

/v1/pos/products
/v1/posts
/v1/sensors
/v1/strains

Health (4)

/health
/health/[provider]/callback
/health/ingest
/health/summary

Commerce (1)

/shop/aliexpress

Audio platform (4)

/audio/[trackId]/progress
/audio/[trackId]/purchase
/audio/[trackId]/rating
/audio/subscribe

Cannabis market and places (4)

/cannabis/market
/cannabis/places
/cannabis/places/report
/cannabis/quotes

Mine sites (2)

/mine/sites
/mine/sites/report

Inbound webhooks (1)

/webhooks/stripe

Core and shared (23)

/cctv/kvs
/comments
/conversations
/debug/auth
/diag
/drone/report
/farm/readings
/games/drone/score
/heartbeat
/link-preview
/media/chat/[...path]
/messages
/metals
/metals/quotes
/notifications
/pos/products.csv
/posts
/quakes
/search
/storage/presign
/support/audio
/webrtc/ice
/wellness/breath/session

Gwave — System Design & Architecture

C Appendix C — Table inventory

All 159 tables defined across the 93 migrations and 48 production patch files, joined by 241 foreign-key relationships. Row level security is enabled on every one of them.

affiliate_clicks
alerts
api_keys
api_logs
audio_audiobook
audio_chapters
audio_entitlements
audio_lyrics
audio_music
audio_plans
audio_podcast
audio_progress
audio_ratings
audio_subscriptions
audio_tracks
audit_logs
automation_rules
blocks
book_progress
book_purchases
books
boost_impressions
boosts
breath_sessions
business_expenses
camera_alerts
camera_clips
camera_group_shares
cannabis_place_reports
cannabis_places
cannabis_quotes
certificates
chess_wager_views
chess_wagers
cohost_guests
cohost_rooms
comments
consents
conversation_participants
conversations
creator_earnings
currency_rates
dating_matches
dating_profiles
dating_swipes
deletion_requests
device_commands
device_tokens
devices
drone_detections
drone_race_scores
family_circles
family_memberships
feature_flags
follows
friendships
game_catalog
game_comments
game_reactions
games
gpay_accounts
gpay_pins
gpay_transactions
group_members
groups
health_connections
health_daily_summary
health_events
health_metrics
health_state
health_vitals
inventory
invoices
job_applications
jobs
lesson_comments
lesson_progress
live_chat_messages
live_gift_events
live_gifts
live_locations
live_products
live_reactions
live_stream_keys
live_stream_presence
live_streams
market_listings
member_locations
member_projects
membership_plans
messages
metal_quotes
mine_site_reports
mine_sites
minerals
notifications
page_followers
pages
payments
phone_otps
podcast_shows
pos_categories
pos_customers
pos_products
post_media
post_views
posts
profiles
ptt_channel_members
ptt_channels
ptt_messages
push_subscriptions
reactions
reel_likes
reel_views
reels
reports
reviews
ride_driver_balances
ride_driver_locations
ride_driver_profiles
ride_ledger
ride_offers
ride_settings
ride_vehicle_types
safety_checkins
sale_items
sale_payments
sales
scene_schedules
scenes
search_queries
sensor_readings
shares
shifts
shop_orders
shop_products
site_settings
sos_alerts
sos_responders
stock_movements
store_members
stores
stories
story_views
strains
subscriptions
support_tickets
teacher_applications
threat_alerts
trackers
typing_scores
user_cameras
webhook_deliveries
webhooks
wellness_items
wifi_networks
wifi_scans
wifi_watchlist

Gwave — System Design & Architecture

D Appendix D — Environment variable reference

Variables split into two classes with different lifecycles. Getting the class wrong is the most common deployment mistake in the project: a build-time variable changed at runtime has no effect, and a runtime secret added as a build argument would be inlined into a public JavaScript bundle.

D.1 Build-time — inlined into the client bundle and the CSP (19 supplied by `deploy.yml`)

> These are public by definition. Changing one requires a rebuild and redeploy; omitting one silently disables the feature it powers.

- `NEXT_PUBLIC_CCTV_APP`
- `NEXT_PUBLIC_CCTV_HLS_ORIGINS`
- `NEXT_PUBLIC_CCTV_PLAYER_ORIGIN`
- `NEXT_PUBLIC_COGNITO_CLIENT_ID`
- `NEXT_PUBLIC_COGNITO_DOMAIN`
- `NEXT_PUBLIC_DATA_API_KEY`
- `NEXT_PUBLIC_DATA_API_URL`
- `NEXT_PUBLIC_GOOGLE_CLIENT_ID`
- `NEXT_PUBLIC_GOOGLE_MAPS_API_KEY`
- `NEXT_PUBLIC_IVS_RECORDING_BASE`
- `NEXT_PUBLIC_LIVEKIT_URL`
- `NEXT_PUBLIC_S3_CDN`
- `NEXT_PUBLIC_SITE_URL`
- `NEXT_PUBLIC_SUPABASE_ANON_KEY`
- `NEXT_PUBLIC_SUPABASE_URL`
- `NEXT_PUBLIC_TURN_CREDENTIAL`
- `NEXT_PUBLIC_TURN_URL`
- `NEXT_PUBLIC_TURN_USERNAME`
- `NEXT_PUBLIC_VAPID_PUBLIC_KEY`

D.2 Runtime — read from `/etc/gwave-web.env` on the host

> Never enters the image. An env-only change needs just `sudo gwave-redeploy` — no CI run, no rebuild.

- `AGORA_APP_CERTIFICATE`
- `AGORA_CUSTOMER_ID`
- `AGORA_CUSTOMER_SECRET`
- `AGORA_RECORDING_S3_ACCESS_KEY`
- `AGORA_RECORDING_S3_BUCKET`
- `AGORA_RECORDING_S3_PREFIX`
- `AGORA_RECORDING_S3_REGION`
- `AGORA_RECORDING_S3_SECRET`
- `ALIEXPRESS_APP_KEY`
- `ALIEXPRESS_APP_SECRET`
- `ALIEXPRESS_TRACKING_ID`
- `APP_JWT_PRIVATE_KEY`
- `APP_JWT_PUBLIC_JWK`
- `AWS_S3_CHAT_BUCKET`
- `AWS_S3_SLIPS_BUCKET`
- `CCTV_MEDIA_API_TOKEN`
- `CCTV_MEDIA_API_URL`
- `CHAT_MEDIA_FALLBACK_PUBLIC`
- `CHIME_CONTROL_REGION`
- `CHIME_MEDIA_REGION`
- `COGNITO_CLIENT_SECRET`
- `COOLIFY_API_TOKEN`
- `COOLIFY_API_URL`
- `COOLIFY_APP_UUID`
- `DATABASE_URL`
- `GIT_COMMIT_SHA`
- `IVS_RECORDING_CONFIG_ARN`
- `IVS_REGION`
- `IVS_RT_ENCODER_CONFIG_ARN`
- `IVS_RT_STORAGE_CONFIG_ARN`
- `LIVEKIT_API_KEY`
- `LIVEKIT_API_SECRET`
- `LIVEKIT_EGRESS_S3_ACCESS_KEY`
- `LIVEKIT_EGRESS_S3_BUCKET`
- `LIVEKIT_EGRESS_S3_PREFIX`
- `LIVEKIT_EGRESS_S3_REGION`
- `LIVEKIT_EGRESS_S3_SECRET`
- `LIVE_SWEEP_SECRET`
- `MUX_TOKEN_ID`
- `MUX_TOKEN_SECRET`
- `MUX_WEBHOOK_SECRET`
- `PROMPTPAY_ID`
- `RIDE_DISPATCH_SECRET`
- `RIDE_GEOCODER_URL`
- `STRIPE_SECRET_KEY`
- `STRIPE_WEBHOOK_SECRET`
- `SUPABASE_JWT_SECRET`
- `TURN_CREDENTIAL`

Gwave — System Design & Architecture

`TURN_URL`
`TURN_USERNAME`
`TWA_PACKAGE_NAME`
`TWA_SHA256_FINGERPRINT`
`TWILIO_ACCOUNT_SID`
`TWILIO_AUTH_TOKEN`
`TWILIO_FROM`
`VAPID_PRIVATE_KEY`
`VAPID_SUBJECT`

D.3 Variables with operational consequences

Variable	Consequence if wrong or missing
`APP_JWT_PRIVATE_KEY / _PUBLIC_JWK`	Every authenticated data call fails; the key id must match the JWKS PostgREST loads.
`SUPABASE_JWT_SECRET`	Must equal the realtime container's `API_JWT_SECRET` and PostgREST's `oct` key, or the Realtime socket will not handshake.
`NEXT_PUBLIC_S3_CDN`	Selects the S3 storage path. Unset falls back to the legacy branch, which production must never take.
`TURN_URL / _USERNAME / _CREDENTIAL`	Served by `/api/webrtc/ice`. Missing means carrier-NAT calls connect and stay silent.
`IVS_RT_ENCODER_CONFIG_ARN / _STORAGE_CONFIG_ARN`	Without them the composition cannot start, so browser broadcasts never reach an HLS URL phones can play.
`LIVEKIT_EGRESS_S3_*`	Currently unset — the reason browser Go Live sessions have no replays.
`LIVE_SWEEP_SECRET`	Authenticates the replay sweeper; falls back to `RIDE_DISPATCH_SECRET`.
`STRIPE_WEBHOOK_SECRET`	Unverified webhooks are rejected, so top-ups and memberships never settle.

End of document. Generated from the `KoNyein/gwave.ai` repository — route counts, table names, foreign keys, workflow steps and environment variables were read out of the source rather than transcribed.